

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Guía técnica

App web y móvil de la Estación meteorológica

Plataforma de Monitoreo y Visualización Meteorológica

Trabajo Terminal 2026-B033

Alumnos:

Rodríguez Reyes Ayax Alexander
Sánchez Leyva Eduardo Samuel
Velazquez Garduño Jorge Nahum

Directores de Trabajo Terminal:

Gómez Mayorga Margarita Elizabeth
Ilizaliturri Flores Ian

6 de noviembre de 2026



Índice general

Presentación **4**
Qué es la aplicación . . . 4
A quién va dirigida . . . 4
Cómo está organizada . . . 5
De dónde sale este documento . . . 5

Visión y especificación 6

Visión **7**
Problema . . . 7
Por qué esta app . . . 7
Objetivos de la app . . . 7
Alcance . . . 8
Actores . . . 8
Restricciones . . . 8

Requerimientos **9**
Requerimientos funcionales . . . 9
Requerimientos no funcionales . . . 10
Diferencias con el cap. 6 de la tesis . . . 12

Casos de uso **13**
Diagrama . . . 13
Especificaciones . . . 13

Interfaz **16**
Pantallas . . . 16
Navegación . . . 16
Contenido de "Ahora" . . . 16
Contenido de "Histórico" . . . 17
Widget de Android . . . 17
Preferencias . . . 18
Sistema visual . . . 18
Accesibilidad (RNF-APP-05) . . . 18

Diseño técnico 19

Arquitectura **20**
Nivel 1 · Contexto . . . 20
Nivel 2 · Contenedores . . . 21
Nivel 3 · Componentes (hexagonal) . . . 21

Despliegue . . . 23

Modelo de dominio **24**
Decisiones del modelo . . . 24
Diagrama de clases . . . 25
Casos de uso (aplicación) . . . 25
Reglas de dominio (funciones puras) . . . 26

Flujos **27**
Dato en vivo (CU-01, CU-02) . . . 27
Histórico (CU-04, CU-05) . . . 28
Cambio de preferencias (CU-06) . . . 28
Estados de la pantalla "Ahora" . . . 29

Datos **30**
Dato en vivo · `wflexp.json` (consola Davis) . . . 30
Histórico · Cloud Function `obtener-datos-clima` . . . 30
Rarezas y dónde se corrigen . . . 31
Escena del cielo . . . 33
Interpretación de variables (RF-APP-04) . . . 33

Implementación y desarrollo 35

Implementación **36**
Tecnologías, en una frase cada una . . . 36
Mapa del repositorio . . . 36
La idea central: escribir la lógica una vez . . . 37
Recorrido 1 · El dato en vivo, de la estación a la tarjeta . . . 39
Recorrido 2 · El histórico . . . 41
Recorrido 3 · Preferencias, tema, idioma y tamaño de letra . . . 42
Recorrido 4 · El widget de Android . . . 42
Detalles de la plataforma que conviene conocer . . . 43
Cómo encontrar algo . . . 43

Entorno de desarrollo **44**
Lo que se necesita . . . 44
Primeros pasos . . . 44
Correr las apps . . . 45
Verificar antes de subir cambios . . . 45
Compilar el APK . . . 46
Problemas comunes . . . 46

Cómo contribuir **47**
Antes de empezar . . . 47
El flujo, paso a paso . . . 47
Lista de revisión de un PR . . . 48
Convenciones de código . . . 48
Cómo escribir pruebas . . . 48
Recetas . . . 49

Calidad y pruebas **51**
Estrategia por capa . . . 51
Cómo se documenta una prueba . . . 51
Cómo correrlas . . . 52
Umbrales . . . 52

Matriz de trazabilidad . . . 52
Accesibilidad (RNF-APP-05) . . . 54

Operación **56**

Despliegue (anexo operativo) **57**
Entornos . . . 57
Diagrama de despliegue . . . 58
Reglas del flujo . . . 58
Infraestructura . . . 58
Configuración . . . 59
Costos estimados (RNF-APP-11) . . . 59
APK de Android (local y a pedido) . . . 59
Rollback . . . 60

Mantenimiento y traspaso **61**
Inventario: de qué depende la app . . . 61
Tareas periódicas . . . 62
Actualizar dependencias . . . 62
Actualizar el SDK de Expo . . . 62
Cambios de configuración frecuentes . . . 63
Cuando algo falla en producción . . . 63
Pendientes conocidos . . . 64

Glosario **65**
Meteorología y la estación . . . 65
Software y arquitectura . . . 65

Registro de decisiones (ADR) **68**
ADR-0001 · Un monorepo con núcleo compartido . . . 68
ADR-0002 · Next.js para web y Expo para móvil, en vez de solo Expo . . . 69
ADR-0003 · npm workspaces en vez de pnpm . . . 69
ADR-0004 · Web en Cloud Run, desplegada desde GitHub Actions . . . 70
ADR-0005 · Móvil: staging y producción por OTA; APK a mano al final . . . 70
ADR-0006 · Gráficas en SVG propio con geometría compartida . . . 70
ADR-0007 · Proxy para el dato en vivo, también para el móvil . . . 71
ADR-0008 · Especificación en `docs/` antes del código y antes del LaTeX . . . 72
ADR-0009 · El arte del diseño se extrae del canvas con un script . . . 72
ADR-0010 · Las animaciones del diseño se describen como datos para móvil . . . 73
ADR-0011 · Widget de Android con react-native-android-widget . . . 73
ADR-0012 · Exportar a Parquet con hyparquet-writer dentro del núcleo . . . 74
ADR-0013 · Caché persistente del histórico por día . . . 75

Notas para la tesis (caps. 6 y 7) **76**
Capítulo 6 · Arquitectura del sistema . . . 76
Capítulo 7 · Implementación . . . 77
Figuras que se reemplazan . . . 78

Presentación

Esta guía técnica describe por completo la aplicación web y móvil de la **Plataforma de Monitoreo y Visualización Meteorológica** de la ESCOM (IPN), desarrollada como Trabajo Terminal 2026-B033. Reúne en un solo documento la especificación, el diseño técnico, la implementación, la forma de contribuir y la operación del sistema, para que el proyecto pueda consultarse, evaluarse y continuarse sin depender de quien lo escribió.

Qué es la aplicación

La estación meteorológica Davis Vantage Vue de la ESCOM mide temperatura, humedad, viento, lluvia y presión cada pocos segundos. Un *pipeline* de datos en Google Cloud (fuera de este repositorio) guarda una lectura cada 15 minutos. La aplicación es la cara visible de ese sistema:

- **Ahora:** las condiciones actuales, actualizadas cada 60 segundos, con una ilustración animada del cielo, el pronóstico de la consola y la explicación de cada variable.
- **Histórico:** gráficas, indicadores y rosa de los vientos de cualquier periodo desde abril de 2026, con descarga de los datos en CSV, JSON o Parquet.
- **Widgets de Android** con la lectura actual en la pantalla de inicio.

Todo en español o inglés, tema claro u oscuro, unidades métricas o imperiales y tres tamaños de letra.

- Web: https://weather-web-nspsuyfpwa-uc.a.run.app
- Código: https://github.com/Samu31Nd/weather_app

A quién va dirigida

Si eres…	Te sirve sobre todo
Profesor o sinodal que revisa el trabajo	Parte I (qué se pidió y para quién) y Parte II (cómo se diseñó); el capítulo de calidad y pruebas, con la matriz de trazabilidad, muestra cómo se verifica cada requerimiento
Estudiante que retoma el proyecto	Empieza por *Implementación* y *Entorno de desarrollo*; después *Cómo contribuir* antes de tu primer cambio
Responsable de operar el sistema	Parte IV: despliegue, mantenimiento y la lista de traspaso
Alguien que solo quiere usar los datos	*Datos* (qué significa cada campo y sus rarezas) y el glosario

No se presupone experiencia con React, Expo ni Google Cloud. Los términos técnicos se explican la primera vez que aparecen y están reunidos en el **glosario** del apéndice. Sí ayuda saber programar en algún lenguaje y conocer lo básico de Git.

Cómo está organizada

- **Parte I · Visión y especificación.** El problema, los objetivos, los requerimientos (`RF-APP-NN`, `RNF-APP-NN`), los casos de uso (`CU-NN`) y la interfaz.
- **Parte II · Diseño técnico.** Arquitectura, modelo de dominio, flujos y el diccionario de datos con sus rarezas (`D1`...`D13`).
- **Parte III · Implementación y desarrollo.** Cómo está construido el código, cómo preparar el entorno, cómo contribuir y cómo se prueba.
- **Parte IV · Operación.** Despliegue, CI/CD, costos, mantenimiento y traspaso.
- **Apéndices.** Glosario, registro de decisiones de arquitectura (ADR) y notas para la tesis.

Los identificadores (`RF-APP-07`, `CU-04`, `D5`, `ADR-0013`) se citan en el código, en las pruebas y en los *pull requests*: buscarlos en el repositorio lleva directo a dónde se implementan y se prueban.

De dónde sale este documento

El PDF se genera a partir de los archivos Markdown de la carpeta `docs/` del repositorio, que son la fuente de verdad y se actualizan junto con el código. Si este PDF y el repositorio no coinciden, gana el repositorio: el PDF se vuelve a generar con `docs/pdf/build.sh`. Los enlaces a archivos del código apuntan a la rama `main` en GitHub.

Visión y especificación

Visión

Problema

La estación Davis Vantage Vue de ESCOM ("Liz") mide cada pocos segundos, y el pipeline en GCP guarda una lectura cada 15 minutos desde mayo de 2026. Aun así, a la comunidad le cuesta usar esos datos:

- La consola solo expone un JSON crudo, en unidades imperiales y por HTTP. No se puede consultar desde un navegador por CORS, y sin contexto los valores no dicen nada a quien no es especialista.
- El histórico vive en una API sin interfaz, que corta en 1000 filas y entrega números como texto.
- Había dos apps a medias (Expo y Next.js) con lógica duplicada, sin pruebas y con errores de datos distintos en cada una. Por ejemplo, máximos y mínimos invertidos, o presión etiquetada como hPa que en realidad estaba en mmHg.

Por qué esta app

Es la capa de **utilización** del sistema: el objetivo específico de *Visualización* de la tesis, más la parte interactiva de *Analítica*. Sin ella, la ingesta, el ELT y la limpieza no llegan a nadie. Convierte las lecturas crudas en información que se entiende al primer vistazo y sirve para estudiar el clima local:

- **Ahora:** ¿qué tiempo hace en ESCOM, cómo se siente y qué viene?
- **Histórico:** ¿cómo se comportó una variable en un periodo? Con descarga de los datos para análisis propio.
- **Interpretación:** qué significa cada variable (punto de rocío, Beaufort, tendencia de presión), para que también le sirva a quien no es especialista.

Objetivos de la app

ID	Objetivo	Objetivo específico de la tesis que atiende
OBJ-1	Mostrar las condiciones actuales en web y móvil, con un retraso máximo de 2 minutos	Visualización (soft real-time)
OBJ-2	Consultar, graficar y resumir el histórico por rangos de fechas	Analítica
OBJ-3	Explicar cada variable en lenguaje llano	Visualización (componente educativo, RF-10 de la tesis)
OBJ-4	Descargar los datos consultados en formatos abiertos (CSV/JSON)	Divulgación
OBJ-5	Mantener una sola lógica, probada, para las dos plataformas	Calidad (mantenibilidad)

Alcance

Incluye:

- App web (Next.js) y app móvil (Expo, Android primero).
- Pantallas "Ahora", "Histórico" y "Carga".
- Escena del cielo animada.
- Preferencias de tema, idioma, unidades y tamaño de texto.
- Descargas.

No incluye:

- **Cuentas ni login.** La app es de solo lectura y los datos son públicos.
- **Predicción numérica** (ARIMA/SARIMA): se investigó en la tesis, pero no se implementa. El único pronóstico es el cualitativo que calcula la consola (`forrule`).
- **Escritura o corrección de datos.** Eso lo hace el pipeline de `proj-files`.
- **Publicación en tiendas de aplicaciones.**
- **Sensores que la estación no tiene:** UV, radiación solar y ET.

Actores

Actor	Quién	Qué necesita
Comunidad ESCOM	Alumnos, docentes y personal	Ver el clima actual y entenderlo rápido, sobre todo desde el móvil
Estudiante / investigador	Quien estudia o usa los datos	Rangos de fechas, gráficas comparables, KPIs y descarga

Actores de sistema (externos): la **consola Davis** (dato en vivo) y la **Cloud Function** `obtener-datos-clima` (histórico).

Restricciones

- Hay que usar las fuentes que ya existen: no se crean APIs de datos nuevas. El único componente de servidor nuevo es el proxy del dato en vivo, dentro de la app web.
- El costo de hosting debe quedar en la capa gratuita, o lo más cerca posible (RNF-09 de la tesis: menos de 30 USD al mes para todo el sistema).
- El diseño visual es el canvas de Claude Design "Estación meteorológica" (ver 07 · Interfaz).

Requerimientos

Los requerimientos se escriben **por capacidad**, no por tarjeta ni por pantalla. Cada uno describe algo que el usuario puede hacer, o una cualidad que se puede medir. El detalle de qué muestra cada pantalla está en 07 · Interfaz.

Columnas:

- **Prio. (MoSCoW):** **M**ust, **S**hould, **C**ould.
- **Refina:** el requerimiento del cap. 6 de la tesis que detalla, si lo hay.
- **Todos aplican a web y móvil por igual.** Que las dos plataformas ofrezcan lo mismo es en sí un requerimiento (RNF-APP-06).

Requerimientos funcionales

ID	Requerimiento	Prio.	Refina
RF-APP-01	**Consultar las condiciones actuales.** El sistema muestra la última lectura de la estación: temperatura y sensación térmica, humedad y punto de rocío, viento (velocidad, ráfaga, dirección y fuerza Beaufort), presión y su tendencia, lluvia (intensidad y acumulados), extremos del día y del mes, amanecer y ocaso, y condiciones interiores	M	RF-08
RF-APP-02	**Mantener la vigencia del dato.** El sistema renueva la lectura cada 60 s y a petición del usuario, indica su antigüedad y avisa si está desactualizada o si la estación no responde. En ese caso conserva la última lectura válida	M	RF-08
RF-APP-03	**Representar el estado del cielo.** El sistema resume las condiciones en una escena visual y en la frase del pronóstico de la consola. Para elegir la escena combina la hora solar, el pronóstico y las mediciones de lluvia, viento y humedad	S	—
RF-APP-04	**Interpretar las variables.** Cada variable tiene una explicación en lenguaje llano de qué es, cómo se lee y qué indica su valor actual	S	RF-10
RF-APP-05	**Informar el estado de la estación.** El sistema muestra la batería, la señal y el tiempo encendida, para que el usuario sepa si puede confiar en el dato. Nunca expone datos internos de red	C	—
RF-APP-06	**Consultar el histórico por periodo.** El usuario elige un periodo con atajos (últimos N días) o un rango de fechas. El sistema obtiene todas las lecturas del periodo aunque superen el límite de la fuente	M	RF-09
RF-APP-07	**Visualizar las series históricas.** El sistema grafica temperatura y humedad, viento (con rosa de los vientos), lluvia (por intervalo y acumulada) y presión. La resolución temporal es ajustable y por defecto se elige una adecuada al periodo	M	RF-08

ID	Requerimiento	Prio.	Refina
RF-APP-08	**Resumir el periodo.** El sistema calcula indicadores por variable: extremos con fecha y hora, media, amplitud, dirección dominante y calma, total y días con lluvia, intensidad máxima y variación de presión	M	—
RF-APP-09	**Exportar los datos consultados.** El usuario descarga en CSV, JSON o Parquet lo que está viendo (periodo, resolución y variables), con las unidades explícitas	S	—
RF-APP-10	**Personalizar la presentación.** El usuario elige las unidades (métrico/imperial), el idioma (es/en), el tema (sistema/claro/oscuro) y el tamaño del texto. Las preferencias se conservan entre sesiones en el dispositivo	M	—

Criterios de aceptación

ID	Criterio
RF-APP-01	Con un `wflexp.json` real de prueba, cada variable aparece en la unidad elegida y coincide con el valor esperado. Un campo ausente (`---`) se muestra como "sin dato", nunca como 0 ni como `NaN`. Los extremos del día y del mes respetan el orden real de los arreglos `hl*`
RF-APP-02	La lectura se renueva cada 60 ± 5 s y al pulsar "Actualizar". Una lectura de más de 2 min muestra el aviso "desactualizado". Si una consulta falla, se ve "sin conexión" junto a la última lectura y su hora
RF-APP-03	Hay un caso de prueba por cada una de las 21 escenas, que la produce desde sus entradas. Las reglas de pronóstico 0–194 dan una frase en es/en. La 193 da "calibrando" y un código inválido da "sin conexión"
RF-APP-04	Toda variable de la pantalla "Ahora" tiene su texto de ayuda en es/en. Los umbrales de interpretación son los documentados en 06 · Datos
RF-APP-05	La interfaz no contiene la MAC, la IP ni el SSID de la estación (lo verifica un test)
RF-APP-06	Un periodo de 30 días se resuelve en varias consultas, sin duplicados y en orden cronológico. Los atajos y los rangos se interpretan en hora de Ciudad de México. Un periodo sin datos lo informa en vez de dibujar gráficas vacías
RF-APP-07	Con datos de prueba conocidos, cada gráfica dibuja los puntos esperados. Los huecos se ven como cortes, no como ceros. La resolución por defecto es 15 min para 1 día, 1 h para 7 días y 1 día para 30 días
RF-APP-08	Los indicadores de un fixture coinciden con valores calculados a mano en el test
RF-APP-09	El archivo tiene una fila por punto de la resolución elegida (a 15 min, una por lectura), solo las variables visibles, la unidad en el nombre de cada columna y los huecos como celdas vacías
RF-APP-10	Cambiar una preferencia actualiza la interfaz sin recargar. Todas las pantallas se ven completas y legibles en español y en inglés, y en tema claro y oscuro. Tras reiniciar la app la preferencia se conserva. El tema manual funciona también en web

Requerimientos no funcionales

ID	Categoría	Requerimiento	ISO 25010	Refina
RNF-APP-01	Corrección de los datos	Las conversiones de unidades y las correcciones de las fuentes (orden de los extremos, presión en mmHg, presión reducida a nivel del mar, código de tendencia, campos ausentes, fechas en hora local) se aplican **en un solo lugar** y se verifican con datos reales de la estación	Adecuación funcional – Corrección	RF-04
RNF-APP-02	Tolerancia a fallos	Los datos ausentes o incompletos nunca interrumpen la interfaz. Las consultas fallidas se reintentan con espera creciente (máx. 3 intentos). Si la estación no está disponible, la app sigue funcionando y lo indica	Fiabilidad – Tolerancia a fallos	RNF-05
RNF-APP-03	Rendimiento	El primer dato en vivo aparece en menos de 3 s (p75) en una red 4G simulada. Un histórico de 7 días se grafica en menos de 2 s	Eficiencia – Comportamiento temporal	RNF-02
RNF-APP-04	Usabilidad	Un usuario sin conocimientos técnicos consulta e interpreta una variable histórica en 3 interacciones o menos. La interfaz sigue el diseño de referencia (tokens de color, tipografía y espaciado) en ambos temas	Usabilidad – Operabilidad / Estética	RNF-10
RNF-APP-05	Accesibilidad	Cumple WCAG 2.2 AA **en el tema claro y en el oscuro**: contraste $\geq$ 4.5:1 en texto y $\geq$ 3:1 en gráficos, objetivos táctiles $\geq$ 44 px, uso completo con teclado en web, descripciones para lector de pantalla en gráficas e íconos, y respeto de la preferencia de movimiento reducido	Usabilidad – Accesibilidad	—
RNF-APP-06	Multiplataforma	Web (de 360 a 1440 px, últimas 2 versiones de los navegadores principales) y móvil (Android e iOS en las versiones que soporta Expo SDK 58) ofrecen las mismas capacidades	Compatibilidad / Portabilidad – Adaptabilidad	RF-08
RNF-APP-07	Internacionalización	La interfaz completa está disponible en **español** (predeterminado) e **inglés**, sin textos fijos en el código: todo sale de catálogos con las mismas claves en ambos idiomas, incluidos el pronóstico, las interpretaciones y los mensajes de error. Fechas y horas se muestran en America/Mexico_City, y números y fechas con el formato del idioma elegido	Usabilidad – Operabilidad	—
RNF-APP-08	Mantenibilidad	Arquitectura hexagonal: la lógica de dominio y los casos de uso viven una sola vez en un núcleo compartido, sin dependencias de React ni de la plataforma, y las dos apps lo consumen sin cambios	Mantenibilidad – Modularidad / Reusabilidad	RNF-07

ID	Categoría	Requerimiento	ISO 25010	Refina
RNF-APP-09	Capacidad de prueba	El núcleo tiene una cobertura de líneas $\geq$ 80 %. Cada RF tiene al menos una prueba en la matriz de trazabilidad, y cada archivo de prueba documenta la regla que verifica y su origen	Mantenibilidad – Capacidad de prueba	—
RNF-APP-10	Seguridad y privacidad	Sin secretos en el código del cliente y con tráfico cifrado entre el cliente y el servidor. El intermediario del dato en vivo solo consulta la estación configurada. No se usan cookies de rastreo y las preferencias solo se guardan en el dispositivo	Seguridad – Confidencialidad / Integridad	RNF-03
RNF-APP-11	Uso de recursos	La app no satura la consola: a lo mucho 1 consulta cada 10 s, sin importar el número de usuarios. No repite consultas al histórico de días ya obtenidos, aun entre sesiones (ADR-0013)	Eficiencia – Utilización de recursos	RNF-09

Diferencias con el cap. 6 de la tesis

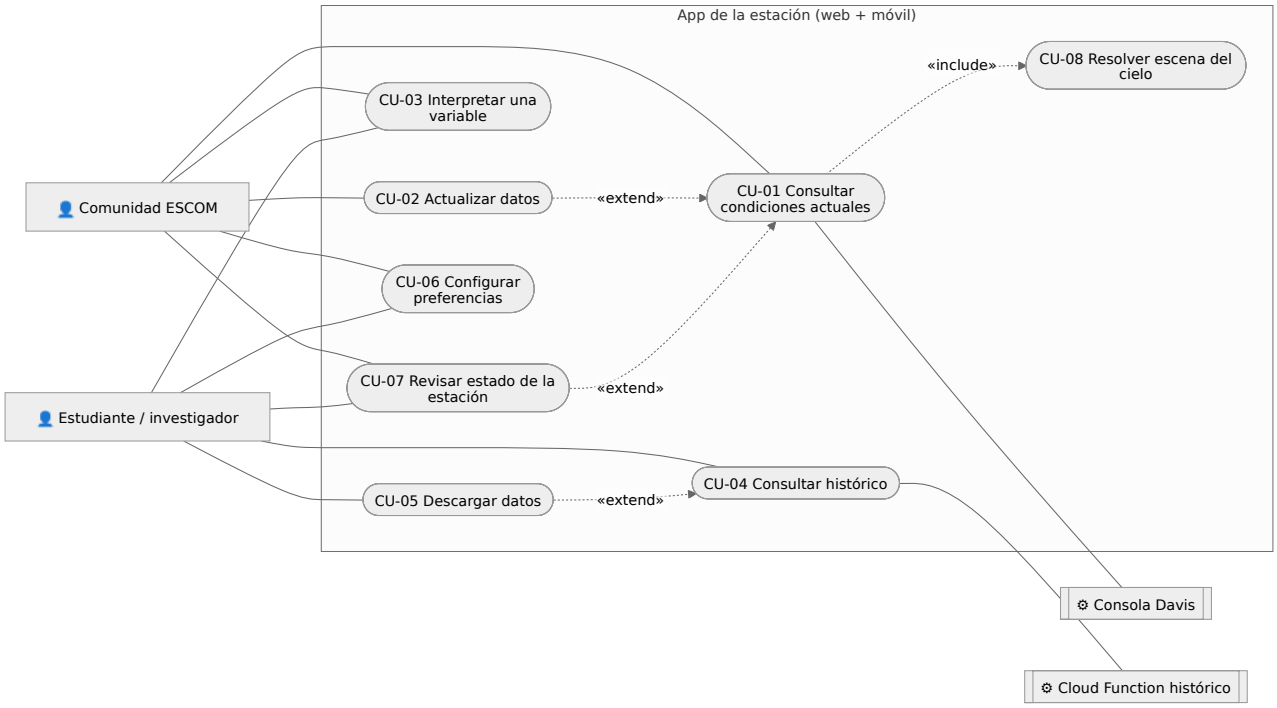
Son notas para actualizar después el cap. 6:

- **RF-08 y RF-09** hablan de "interfaz web". Ahora aplican a **web y móvil** (RNF-APP-06).
- **RF-10** (componente educativo, prioridad Baja) pasa a ser RF-APP-04, con prioridad Should, y queda integrado en cada variable en lugar de ser un módulo aparte.
- La tesis no tiene requerimientos para:
  - el dato en vivo (RF-APP-01 a RF-APP-03);
  - la personalización (RF-APP-10);
  - la accesibilidad (RNF-APP-05);
  - la internacionalización (RNF-APP-07).
- **RNF-APP-01** traslada a la app la idea de RF-04 (validación y limpieza): las correcciones de datos que el pipeline no hace se aplican en el núcleo.

Casos de uso

Mermaid no tiene diagrama UML de casos de uso. Se usa un `flowchart` con esta convención: los actores son nodos rectangulares fuera del sistema, los casos de uso son óvalos dentro del recuadro y `«include»`/`«extend»` van como etiquetas en las flechas.

Diagrama



El investigador también puede hacer todo lo que hace la comunidad (CU-01 y CU-02). Esa herencia no se dibuja para no saturar el diagrama.

Especificaciones

CU-01 Consultar condiciones actuales

- **Actor:** Comunidad ESCOM (y cualquier actor).
- **Cubre:** RF-APP-01, 02, 03.
- **Precondición:** ninguna; no hay login.
- **Flujo principal:**

1. El usuario abre la app.
2. Se muestra la pantalla de carga.
3. El sistema consulta el dato en vivo: en web y en móvil a través del proxy `/api/live` (en desarrollo, el móvil puede ir directo a la consola).
4. El sistema lo convierte a las unidades elegidas y resuelve la escena del cielo (CU-08).
5. Se muestran las tarjetas.
6. Cada 60 s el sistema repite desde el paso 3 sin bloquear la UI.

- **Flujos alternos:**
  - **A1. Sin conexión en la primera carga.** Se muestra la escena "Sin conexión" con un botón para reintentar.
  - **A2. Falla una consulta posterior.** Se conserva la última lectura y se muestra el aviso "sin conexión" junto con la hora del dato.
  - **A3. Dato de más de 2 min.** Aviso "dato desactualizado".
  - **A4. Campo ---.** La tarjeta muestra "sin dato" y las demás siguen normales.
- **Postcondición:** el usuario ve un dato de menos de 2 min, o sabe que no está disponible.

CU-02 Actualizar datos

- **Actor:** Comunidad ESCOM.
- **Cubre:** RF-APP-02.
- **Flujo:**
  1. El usuario pulsa "Actualizar".
  2. El botón pasa a "Actualizando…" y queda deshabilitado.
  3. Se hace una consulta (CU-01, paso 3).
  4. El contador de antigüedad vuelve a 0.
- **Alterno:** si falla, se aplica A2 de CU-01.

CU-03 Interpretar una variable

- **Actores:** Comunidad ESCOM, Estudiante/investigador.
- **Cubre:** RF-APP-04; RNF-APP-04.
- **Flujo:**
  1. El usuario abre la ayuda de una tarjeta.
  2. El sistema muestra qué es la variable, cómo leerla y qué indica el valor actual. Por ejemplo: "punto de rocío a 1 °C de la temperatura: probable neblina".

CU-04 Consultar histórico

- **Actor:** Estudiante / investigador.
- **Cubre:** RF-APP-06, 07, 08; RNF-APP-03, 04.
- **Flujo principal:**
  1. El usuario abre "Histórico".
  2. El sistema propone los últimos 7 días con resolución por hora.
  3. El usuario cambia el periodo o la resolución (opcional).
  4. El sistema toma de la caché los días ya consultados, pide los que faltan en consultas de 10 días o menos, los une, los ordena y los remuestrea.
  5. El sistema calcula los KPIs y dibuja las gráficas por variable.
- **Flujos alternos:**
  - **A1. Rango sin datos** (por ejemplo, un día perdido, como el 2026-09-07). El sistema muestra "sin datos en este periodo" y no dibuja gráficas vacías.
  - **A2. El rango incluye hoy o ayer.** El sistema avisa que el histórico llega con un día de retraso.

  - **A3. Falla la API.** El sistema muestra un error con opción de reintentar y conserva la consulta anterior.
- **Postcondición:** las gráficas y los KPIs corresponden exactamente al periodo elegido.

CU-05 Descargar datos

- **Actor:** Estudiante / investigador.
- **Cubre:** RF-APP-09.
- **Precondición:** hay una consulta de histórico con datos.
- **Flujo:**
  1. El usuario elige CSV, JSON o Parquet.
  2. El sistema genera el archivo con las filas del periodo, con la unidad en el nombre de la columna.
  3. El navegador lo descarga, o en móvil se abre el menú de compartir.

CU-06 Configurar preferencias

- **Actores:** todos.
- **Cubre:** RF-APP-10.
- **Flujo:**
  1. El usuario abre ajustes.
  2. Cambia el tema, el idioma, las unidades o el tamaño de texto.
  3. La UI se actualiza al instante.
  4. La preferencia se guarda en el dispositivo.

CU-07 Revisar estado de la estación

- **Actores:** Comunidad ESCOM, Estudiante/investigador.
- **Para qué:** saber si se puede confiar en el dato. Por ejemplo, con batería baja o mala señal las lecturas pueden atrasarse o faltar.
- **Cubre:** RF-APP-05.
- **Flujo:**
  1. En "Ahora", el pie de la pantalla muestra la batería, la señal y el tiempo encendida.
  2. Si la batería o la señal están por debajo de un umbral, el valor se resalta.
- **Nota:** no se muestran datos de red internos.

CU-08 Resolver escena del cielo

- **Actor:** el sistema, incluido en CU-01.
- **Cubre:** RF-APP-03.
- **Flujo:** se aplican, en orden, las capas de la regla `skyMap` (06 · Datos).

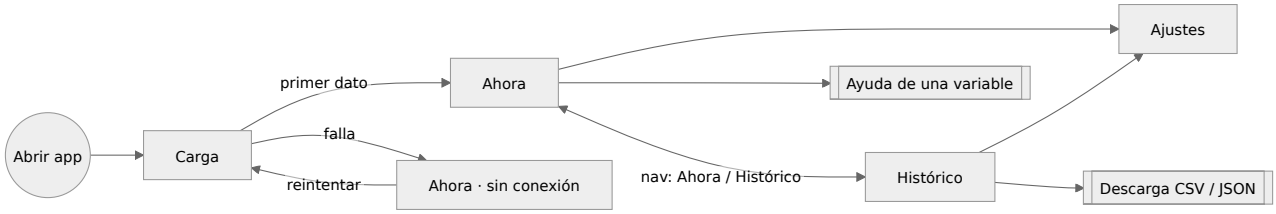
Interfaz

El diseño de referencia es el canvas de Claude Design "Estación meteorológica" (https://claude.ai/artifact/20bb9e0e-2d1d-4854-89ee-aefb52c44c34). Cada pantalla tiene un artboard en tema oscuro y otro en tema claro. Los datos que muestran los artboards son simulados.

Pantallas

Pantalla	Web	Móvil	Casos de uso
Carga	`LoadingWeb`, `LoadingWebClaro`	`LoadingMobile`, `LoadingMobileClaro`	CU-01
Ahora	`Dashboard`, `DashboardClaro`	`MobileNow`, `MobileNowClaro`	CU-01, 02, 03, 07
Histórico	`Main`, `MainClaro`	`MobileHistory`, `MobileHistoryClaro`	CU-04, 05
Ajustes	Controles en la cabecera	Hoja modal `m.settings`	CU-06
Escena del cielo	21 artboards `Sky*` (640×220)	Los mismos, adaptados	CU-08

Navegación



- **Web:** cabecera fija con el nombre de la estación, el estado "en vivo", la navegación, el botón de actualizar, unidades, idioma, tema y tamaño de texto.
- **Móvil:** barra inferior (Ahora / Histórico) y un botón de ajustes que abre una hoja con los mismos controles.
- **En el histórico móvil**, las secciones (temperatura, viento, lluvia, presión) son acordeones, con un control para abrir o cerrar todas.
- **Arranque del APK:** Android 12+ siempre pinta una pantalla del sistema antes de la app. Va sin ícono y con el color de arriba del cielo de la carga (`sky1`, claro u oscuro según el sistema), así que se ve directo la carga.

Contenido de "Ahora"

Va en el mismo orden en las dos plataformas. La web lo acomoda en una cuadrícula y el móvil en una columna.

1. **Hero.**
   - Escena del cielo con su etiqueta.
   - Temperatura exterior.
   - Sensación térmica y su motivo.
   - Rango del día/mes.
   - Humedad, viento, lluvia de hoy y presión.
2. **Pronóstico** de la consola.
3. **Sol:** amanecer, mediodía y ocaso.
4. **Viento:** compás, Beaufort y velocidades.
5. **Presión:** valor, tendencia y variación del día.
6. **Humedad:** rango y punto de rocío.
7. **Interior:** temperatura, humedad y confort.
8. **Lluvia:** intensidad y acumulados.
9. **Estado de la estación:** batería, señal y tiempo encendida.

Contenido de "Histórico"

- **Controles:** periodo (atajos y "últimos N días" o rango personalizado), resolución, la opción de mostrar el interior y la descarga.
- **Secciones**, cada una con KPIs y gráficas:
  - Temperatura: media con banda y humedad.
  - Viento: velocidad y ráfaga, más la rosa de los vientos.
  - Lluvia: barras por intervalo y acumulado.
  - Presión: valor, banda y tendencia.
- En web se puede elegir qué variables se muestran; en móvil cada sección es un acordeón (abre la temperatura) y la descarga incluye las secciones abiertas.
- El periodo termina a la medianoche de hoy en CDMX: el histórico llega con un día de retraso (D7). Hay datos desde el 19-abr-2026, así que "últimos N días" se recorta a lo disponible.
- La resolución se limita a lo que da de 4 a 800 puntos; las demás aparecen deshabilitadas con el motivo.
- La presión usa las unidades de las preferencias (hPa o inHg). El selector mmHg/hPa del artboard no se implementó: mmHg ya aparece como lectura alterna en "Ahora".
- La descarga web baja el archivo; la móvil lo entrega por la hoja de compartir del sistema.

Widget de Android

Solo en el APK (artboards `WidgetsClaro`/`WidgetsOscuro`, ADR-0011).

Tamaño	Contenido
2×2	Ícono del clima, "Liz · hora de la lectura", temperatura, condición y un dato contextual
4×2	Lo mismo + sensación térmica, máx/mín y una columna con humedad, viento y lluvia de hoy; ↻ para actualizar

- **Dato contextual del 2×2:** ráfaga $\geq$ 29 km/h → ráfaga; lloviendo → lluvia de hoy; neblina → humedad; sin conexión → hora de la última lectura; en otro caso, máx/mín del día.
- **Sin conexión:** si la consola no responde, el widget dibuja completa la última lectura guardada (la guardan la app y el widget en `wx-last-reading`, sin importar su antigüedad), con la temperatura en gris y "Última: hora" (con la fecha corta si no es de hoy) como dato contextual. El estado "sin conexión", con "—", queda solo si nunca hubo una lectura.

- **Ícono:** 7 versiones fijas que resumen las 21 escenas (despejado, noche, parcial, lluvia, tormenta, neblina, sin conexión), sobre un fondo sólido tintado por el clima.
- **Tema e idioma:** sigue el tema del sistema; el idioma y las unidades salen de las preferencias de la app.
- **Fuente:** la del sistema, sin fijar familia, y respeta el tamaño de letra del usuario.
- **Actualización:** cada 30 min (mínimo de Android), al tocar ↻ y cada vez que la app recibe un dato nuevo. Tocar el resto del widget abre la app.
- **Accesibilidad:** el texto secundario del tema claro se oscureció a `#5b5d70` para llegar a 4.5:1 sobre todos los fondos (el diseño usa `#6c6f85`).

Preferencias

Preferencia	Valores	Por defecto	Clave en almacenamiento	Efecto
Tema	sistema / claro / oscuro	sistema	`wx-theme`	Web: clase en `<html>` y variables CSS (ADR-0002). Móvil: paleta de `@tt/tokens` según el tema, con `useColorScheme` para "sistema"
Idioma	es / en	es	`wx-lang`	Catálogo de `packages/i18n`. Web: además `<html lang>`
Unidades	métrico / imperial	métrico	`wx-units`	Conversión al mostrar
Tamaño de texto	s / m / l	m	`wx-size`	Web: `zoom` del contenedor `.wx-zoom`. Móvil: `Txt` multiplica cada tamaño (0.9 / 1 / 1.15) y React Native aplica además el tamaño de letra del sistema

Sistema visual

- **Tipografía:** IBM Plex Sans para el texto y IBM Plex Mono para cifras y unidades.
- **Tokens** en `packages/tokens`: paleta clara y oscura (Catppuccin Latte y Mocha) tomada de los artboards. En tema claro el texto de color usa variantes `*-text` más oscuras, para alcanzar 4.5:1.
- **Escenas del cielo** en `packages/sky-art`, extraídas de los artboards `Sky*` con un script (ADR-0009).
- **Iconos:** de trazo, sin emoji.
- **Gráficas:** SVG propio, con geometría compartida desde `packages/core/charts` (ADR-0006).
- **Movimiento:** las escenas y las transiciones se apagan con `prefers-reduced-motion` o con la opción equivalente del sistema.

Accesibilidad (RNF-APP-05)

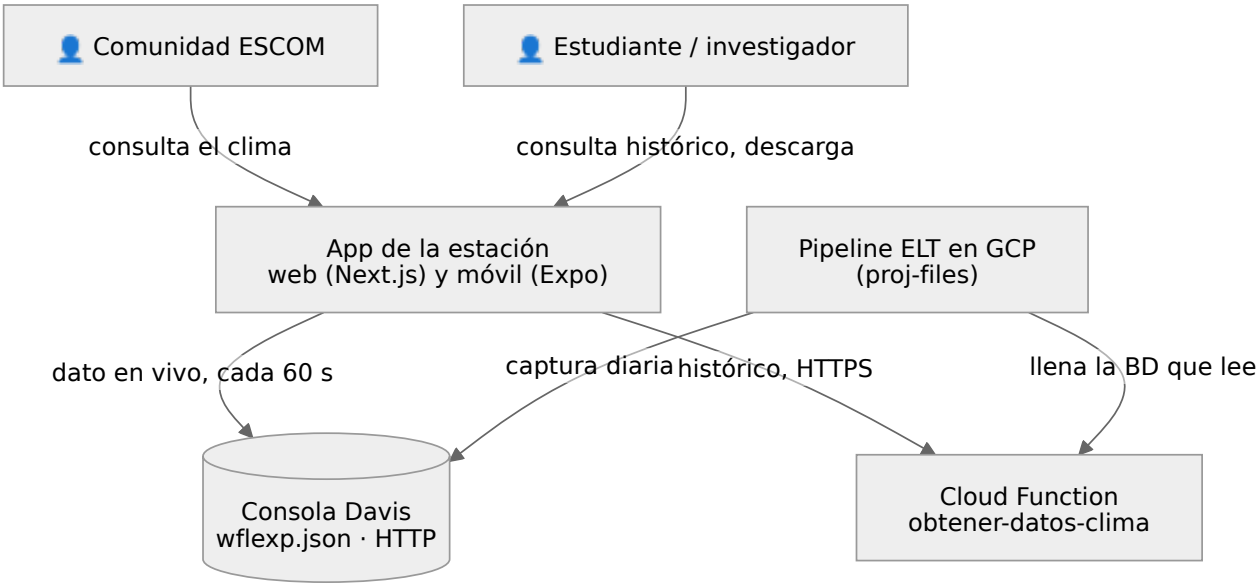
- Contraste $\geq$ 4.5:1 en texto, también en las etiquetas grises de 11 px. Hay que revisarlo en **los dos** temas.
- Botones y enlaces reales: `<button>`/`<a>` en web, `Pressable` con `accessibilityRole` en móvil. `aria-label` en los botones que solo tienen ícono.
- Cada gráfica tiene un resumen textual (`role="img"` + `aria-label`, o `accessibilityLabel`), con el rango y los extremos.
- Los colores que deben distinguirse también difieren en luminosidad, no solo en tono. Se prefiere azul/naranja a rojo/verde.
- Objetivos táctiles $\geq$ 44 px.

Diseño técnico

Arquitectura

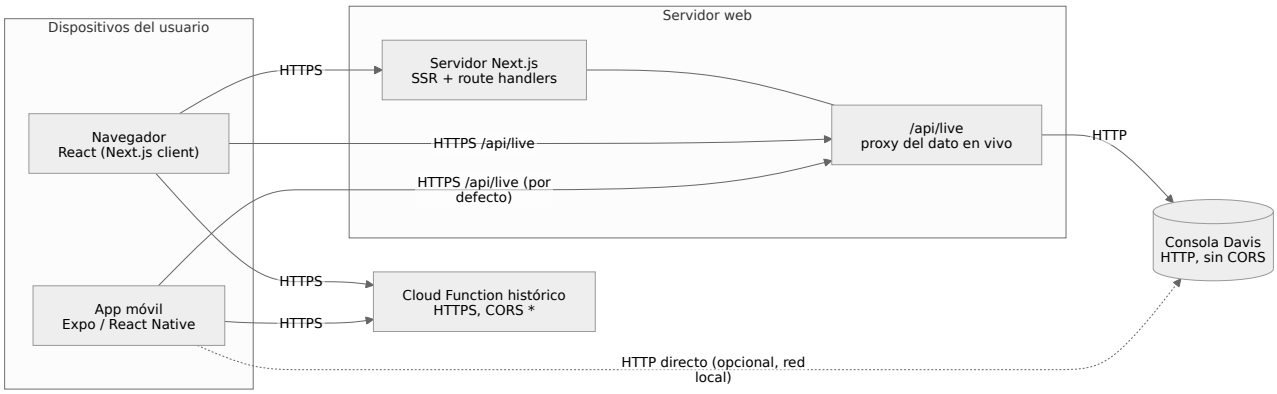
Se sigue el modelo C4 de forma ligera: contexto → contenedores → componentes. La regla central es la **arquitectura hexagonal**: el dominio no conoce React, HTTP ni almacenamiento, y todo eso entra por puertos.

Nivel 1 · Contexto



La app **solo lee**. El pipeline ELT (fuera de este repo) es el único que escribe en la base de datos.

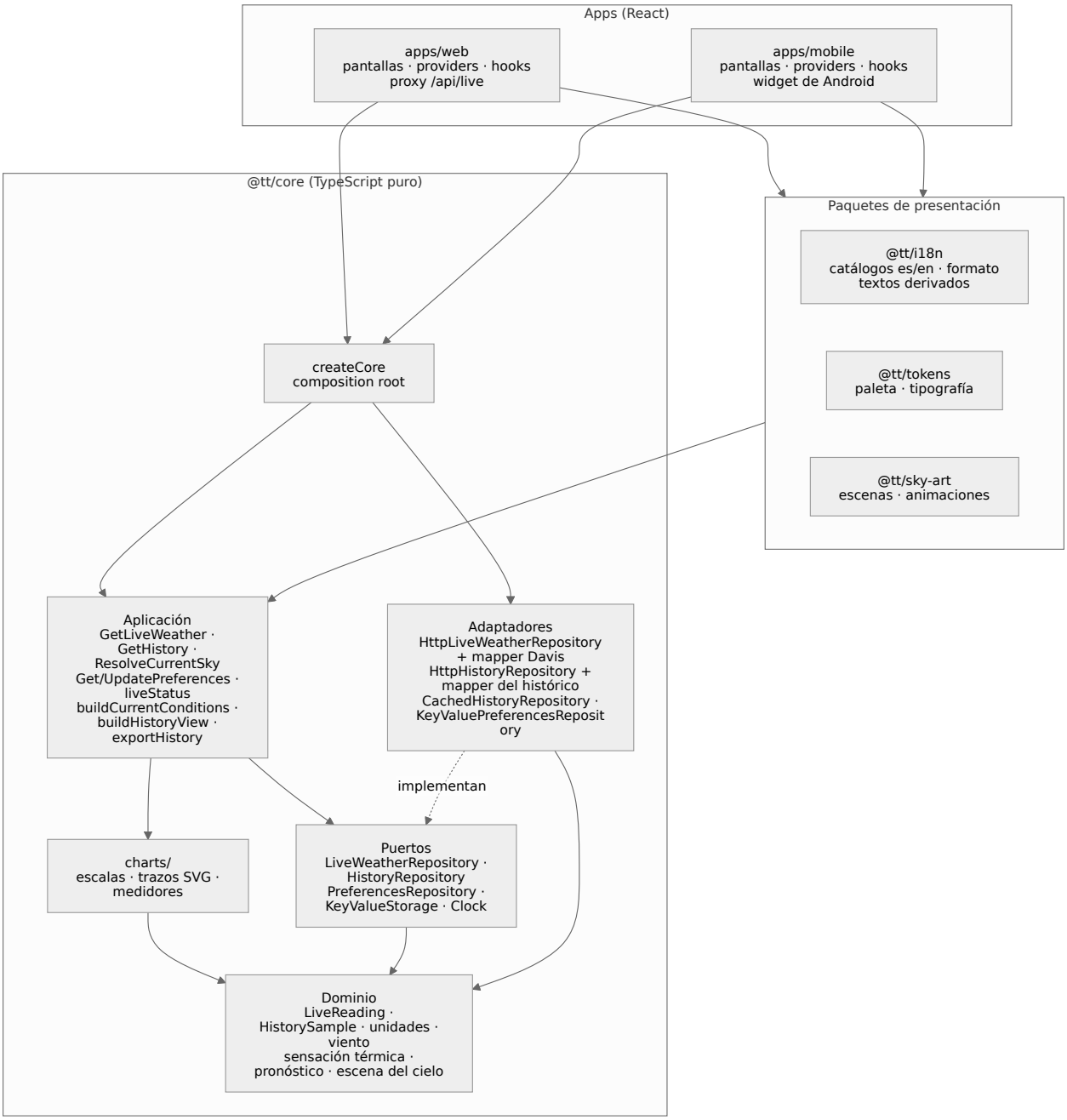
Nivel 2 · Contenedores



- **Por qué hay proxy.** La consola solo habla HTTP y no manda cabeceras CORS. Un navegador servido por HTTPS no puede llamarla. El proxy además oculta la IP de la consola y le baja la carga con caché de unos segundos (ADR-0007).
- **El histórico va directo a la Cloud Function.** Ya tiene CORS abierto, así que no hay que pasarlo por el servidor.
- **El móvil usa el proxy por defecto.** Así evita configurar excepciones de *cleartext* en Android e iOS y funciona fuera de la red del IPN. La conexión directa queda como opción por variable de entorno.

Nivel 3 · Componentes (hexagonal)

Las flechas se leen "depende de". Todas apuntan hacia el núcleo, y dentro de él, hacia el dominio.



- **Plataforma.** Las apps solo aportan el almacén clave-valor (`localStorage` en web, AsyncStorage en móvil) que `createCore` usa para las preferencias y la caché del histórico, y la URL de cada fuente.
- **El proxy `/api/live`** no usa el núcleo: reenvía el JSON de la consola sin datos de red. El mapper corre en el cliente, igual que con la consola directa.
- **Recorrido por el código** de cada flujo, con los nombres de archivo: 12 · Implementación.

Regla de dependencias

Desde \ hacia	dominio	aplicación	puertos	adaptadores	React / plataforma
dominio	✓	✗	✗	✗	✗
aplicación	✓	✓	✓	✗	✗

Desde \ hacia	dominio	aplicación	puertos	adaptadores	React / plataforma
adaptadores (`core`)	✓	✗	✓	✓	✗ (solo `fetch` estándar)
apps	✓	✓	✓	✓	✓

La prueba `packages/core/test/architecture.test.ts` lo hace cumplir: lee cada archivo de `packages/core/src` y falla si importa algo que no sea un archivo propio (nada de `react`, `react-native`, `next`, `expo` ni librerías, salvo `hyparquet-writer`, aprobada por ADR-0012), o si el dominio importa de las otras capas.

Composición (DI)

Cada app arma sus dependencias en un **composition root** (`apps/web/lib/core.ts`, `apps/mobile/src/lib/core.ts`): `createCore({ liveUrl, historyUrl, storage })` devuelve los casos de uso ya conectados. Un provider de React los reparte con `useCore()`. En los tests, `createCore` recibe dobles (`liveRepository`, `historyRepository`, un reloj fijo) en lugar de los adaptadores reales.

Despliegue

La arquitectura no depende de dónde se publique cada app. El servidor web solo necesita un runtime de Node que pueda hacer HTTP saliente. El hosting y la entrega actuales se describen en el anexo operativo 09 · Despliegue.

Modelo de dominio

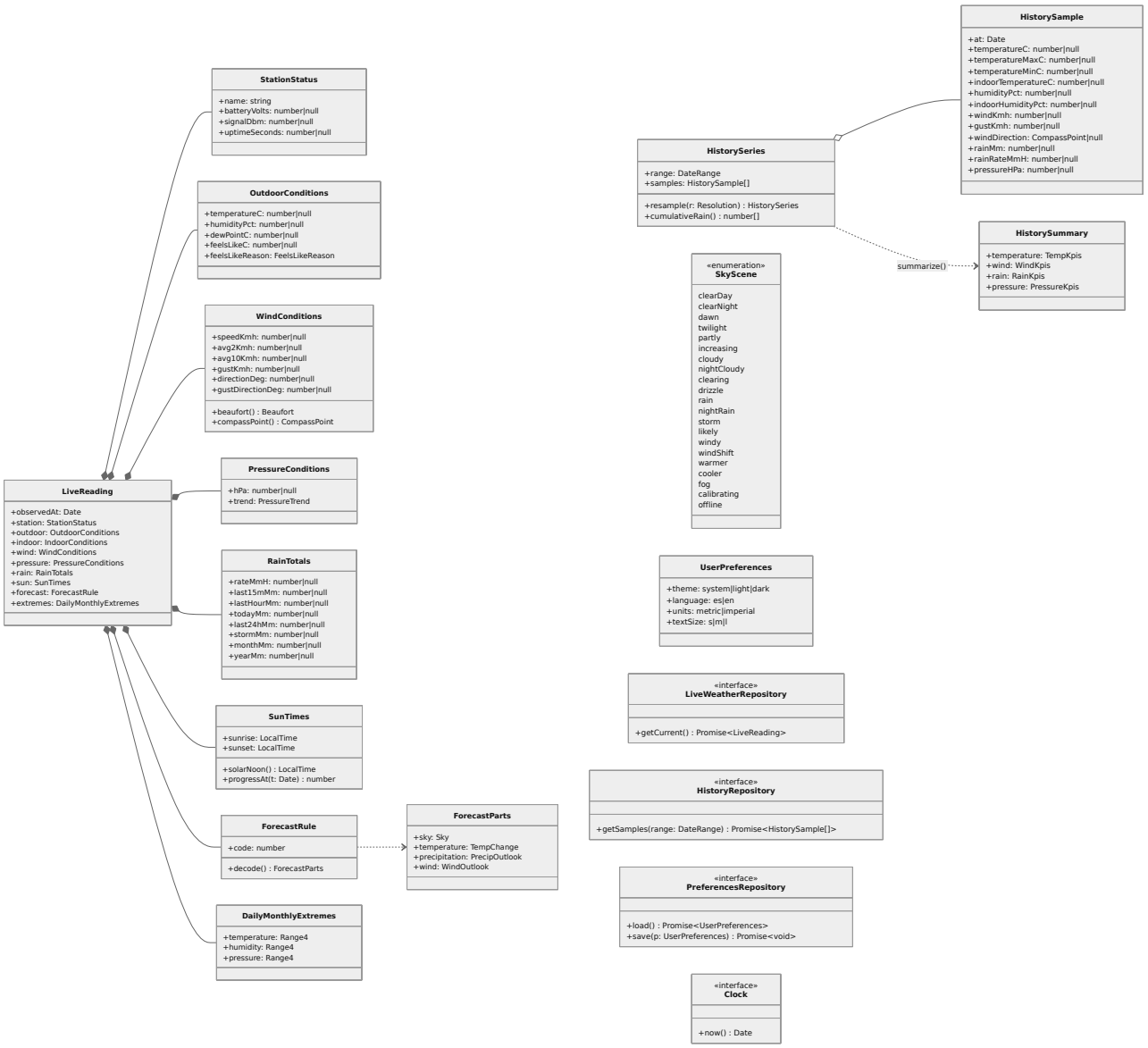
Este modelo une los dos `WeatherEntity` que existen hoy: `apps/mobile/src/core/domain/models/WeatherEntity.ts` y `apps/web/lib/domain/weather.entity.ts`. Corrige sus errores y separa el dato en vivo del histórico.

Decisiones del modelo

- **Unidad canónica interna: métrico SI de la app** (°C, km/h, mm, hPa). Los mappers convierten al entrar. La UI convierte a imperial solo para mostrar. Así el histórico (ya métrico) y el dato en vivo (imperial) se comparan sin conversiones dispersas.
- **Presión.** La consola entrega inHg y el histórico mmHg. Se guarda en hPa. La UI ofrece hPa y mmHg en métrico, e inHg en imperial.
- **"Sin dato" es `null`, nunca 0.** Los mappers traducen `---`, `""` y `null` a `null`.
- **Las fechas son instantes** (`Date`) creados en America/Mexico_City (UTC−6 fijo) desde `fecha` + `hora` o desde `loctime`, nunca desde `utctime` (sale 1 h atrasado, rareza D12).
- **Implementación:** `packages/core/src`. Los nombres de campos finales (`seaLevelHPa`, `extremes.*.today/month/year`, `avg2MinKmh`, ...) están en `domain/live-reading.ts` y `domain/history.ts`; los diagramas de abajo son la vista general.

Diagrama de clases

Va en dos partes que no se relacionan entre sí: la lectura en vivo, y el histórico con las escenas, las preferencias y los puertos.



Casos de uso (aplicación)

Caso de uso	Firma	Depende de	RF
`GetLiveWeather`	`()` → `Promise<LiveReading>`	`LiveWeather Repository`	RF-APP-01, 02
`ResolveCurrentSky`	`(reading: LiveReading | null)` → `SkyScene`, con la hora del `Clock`	reglas de dominio	RF-APP-03
`GetHistory`	`(range: DateRange)` → `Promise<History Sample[]>`: parte en tramos de 10 días o menos, une, quita duplicados y ordena	`History Repository`	RF-APP-06

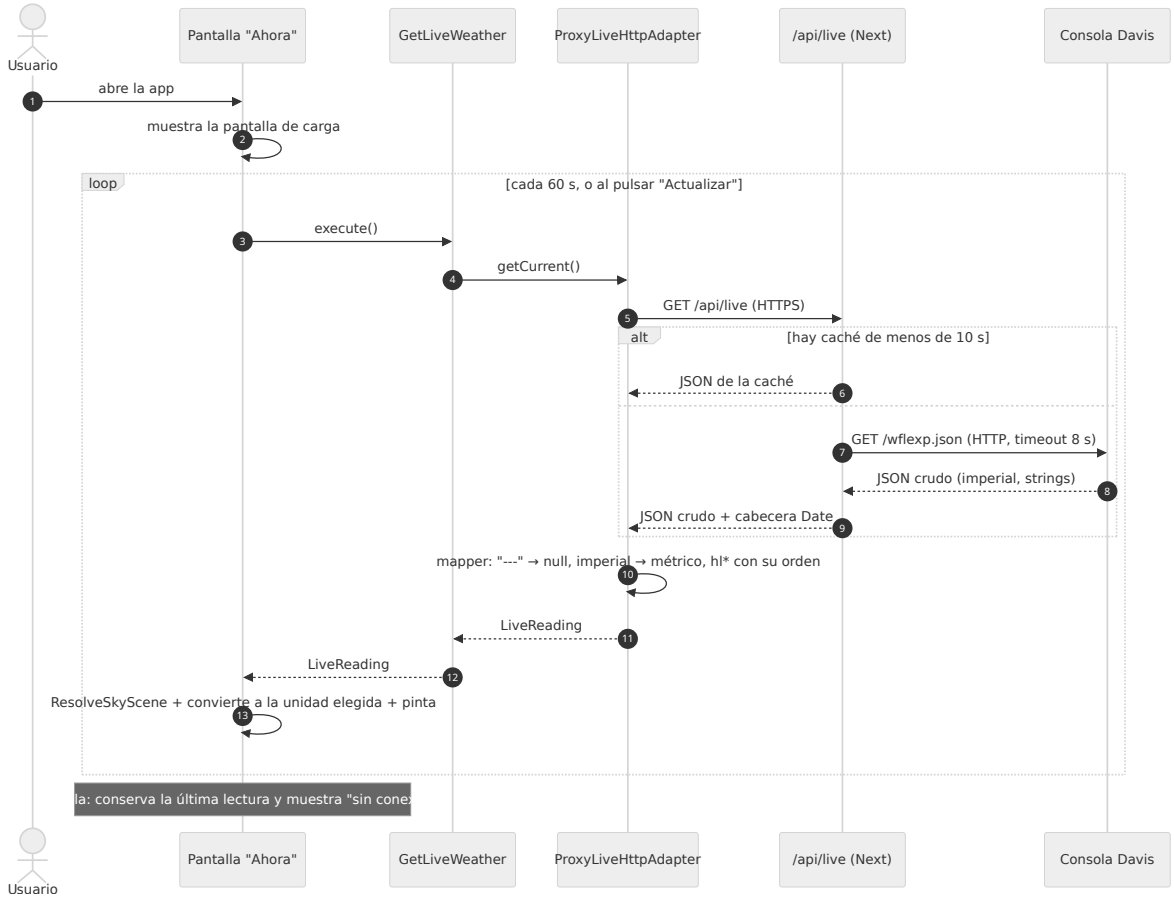
Caso de uso	Firma	Depende de	RF
`BuildChartSeries`	`(series, resolution, units) → ChartSeries`	dominio + `charts/`	RF-APP-07
`SummarizeHistory`	`(series) → HistorySummary`	dominio	RF-APP-08
`ExportHistory`	`(series, format: csv|json|parquet, units) → Blob-like string`	dominio	RF-APP-09
`GetPreferences` / `UpdatePreferences`	`() → Promise<UserPreferences>` / `(patch) → Promise<UserPreferences>`	`PreferencesRepository`	RF-APP-10

Reglas de dominio (funciones puras)

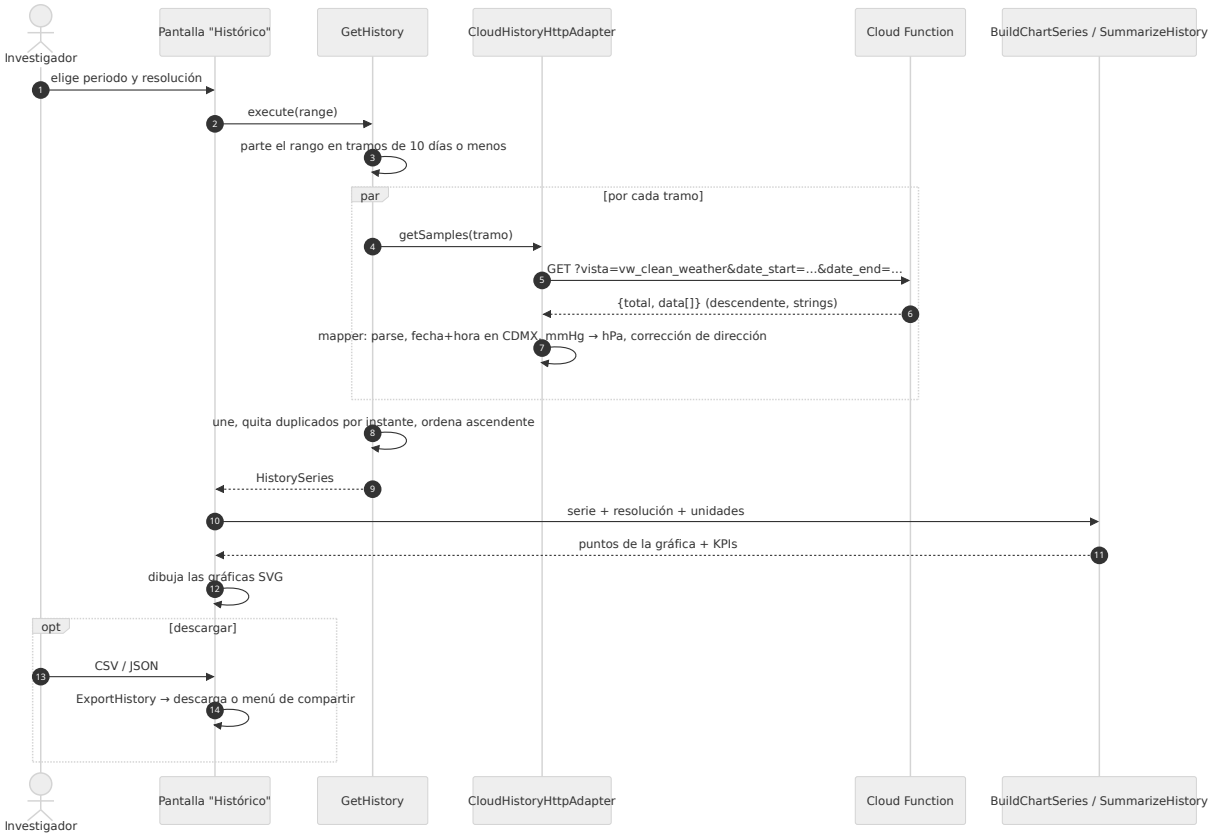
Regla	Definición	Fuente
Sensación térmica	`heat ?? chill ?? temperatura`. El motivo es "calor y humedad", "viento" o "igual que la temperatura"	`realtime-data-schema.md`
Punto de rocío	Lo da la consola (`dew`). Se recalcula con Magnus solo si falta	Meteorología estándar
Beaufort	Tabla oficial en km/h (0: <1, 1: 1–5, …, 12: ≥ 118)	OMM
Rosa de 16 rumbos	`round(deg / 22.5) mod 16`	Convención
Mediodía solar	Punto medio entre amanecer y ocaso	Diseño (`sun.noon`)
Escena del cielo	Tabla de decisión en 06 · Datos	Nota `skyMap` del canvas
Pronóstico	`ForecastDecoder` (195 reglas) → `ForecastParts` → frase por idioma	`apps/mobile/src/core/domain/utils/ForecastDecoder.ts`

Flujos

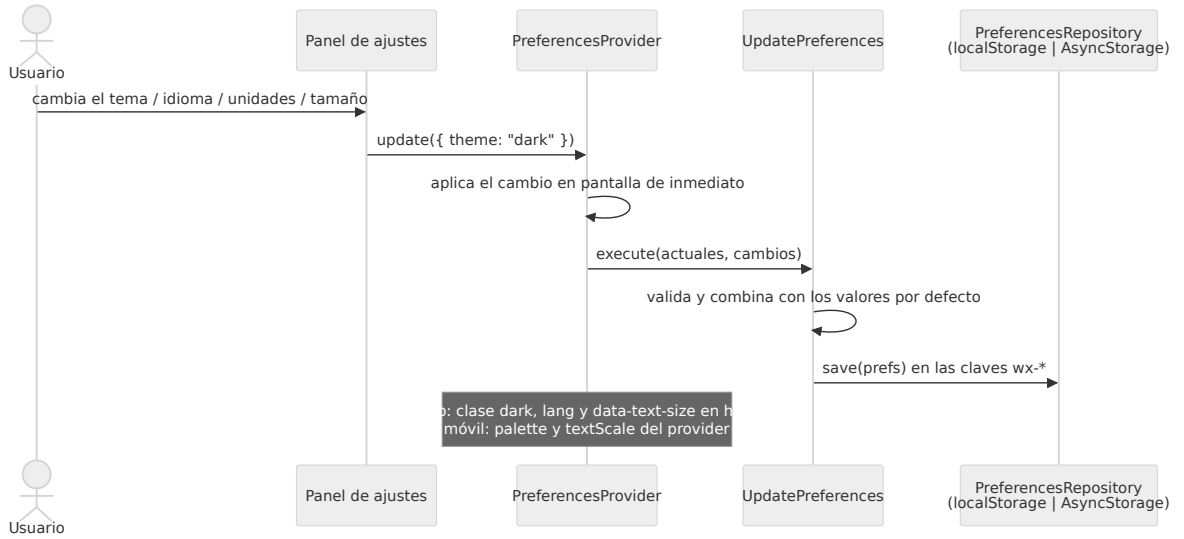
Dato en vivo (CU-01, CU-02)



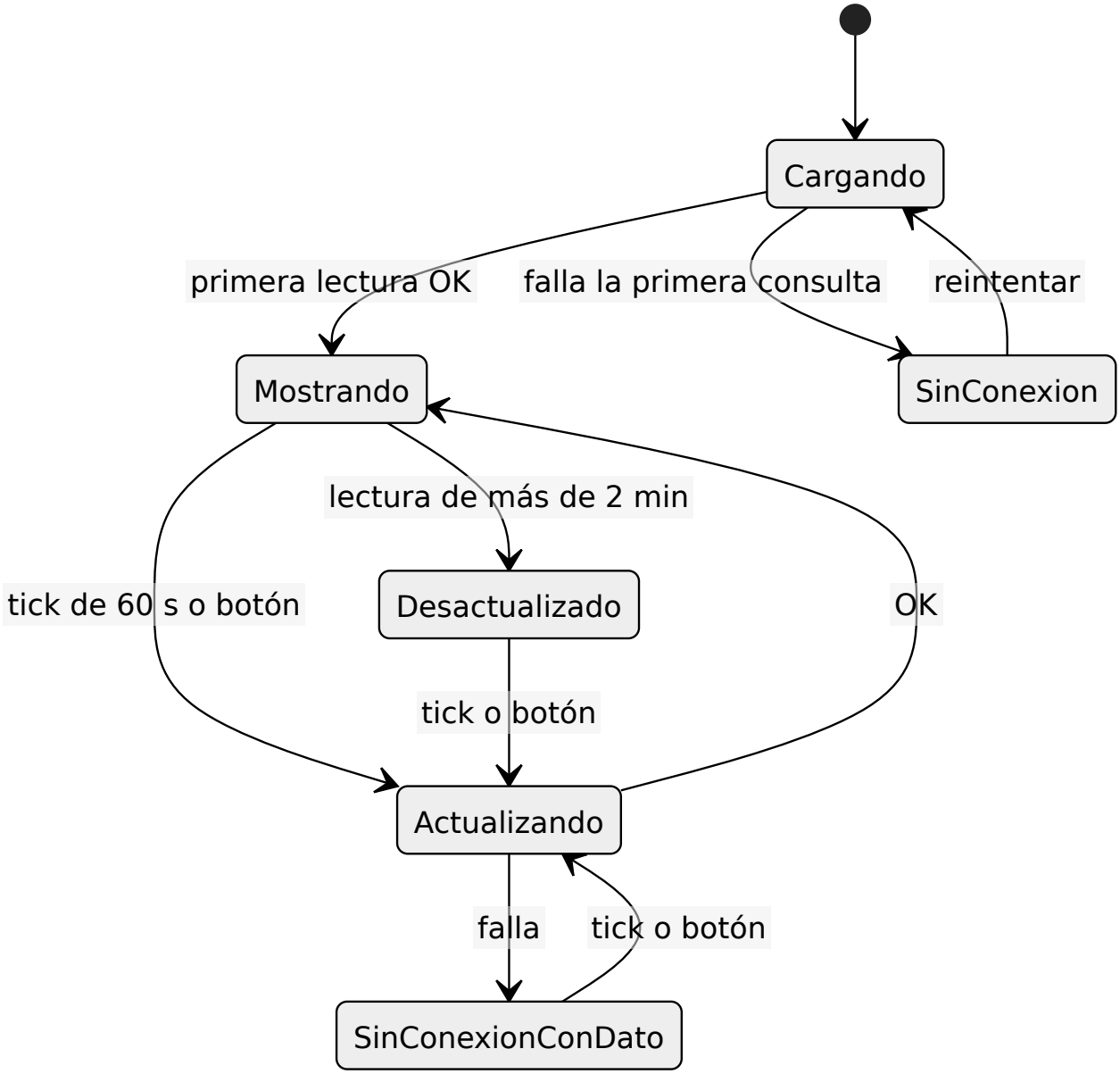
Histórico (CU-04, CU-05)



Cambio de preferencias (CU-06)



Estados de la pantalla "Ahora"



Datos

Hay dos fuentes y las dos son de solo lectura. El detalle de campos con ejemplos está en `apps/web/docs/realtime-data-schema.md` y `apps/web/docs/historical-data-schema.md`. Este documento resume lo que la app necesita y **corrige lo que cambió** desde entonces.

Verificado contra los endpoints reales el 2026-10-21 y el 2026-10-24 (fase 1).

Dato en vivo · `wflexp.json` (consola Davis)

- **Transporte:** HTTP sin CORS. Se consulta por el proxy `/api/live` (ver ADR-0007).
- **Formato:** una foto del instante actual, no una serie. Unidades **imperiales**, casi todo como *string*, y `---` cuando no hay dato.

Grupo	Campos que usa la app	Unidad de origen → canónica
Exterior	`tempout`, `humout`, `dew`, `chill`, `heat`	°F → °C · %
Interior	`tempin`, `humin`	°F → °C · %
Viento	`windspd`, `windavg2`, `windavg10`, `gust`, `winddir`, `gustdir`	mph → km/h · grados
Presión	`bar`, `bartr`, `hlbar`	inHg → hPa
Lluvia	`rainr`, `rain15`, `rain1h`, `raind`, `rain24`, `storm`, `rainmon`, `rainyear`	in, in/h → mm, mm/h
Sol	`sunrt`, `sunst`	`"H:MM"` hora local
Pronóstico	`forrule` (0–194), `foreico`	código
Extremos	`hltempout`, `hlhumout`, `hlbar`, … (8 posiciones: hoy, mes y año; ver D1)	igual que su variable
Estación	`stnname`, `bat`, `rssi`, `uptime`, `loctime`	V · dBm · s · epoch

Datos que no se usan:

- `mac`, `ip` y `ssid` son datos internos y no se muestran nunca (RF-APP-05).
- `uv`, `solar`, `thsw`, `et*`, `xt*`, etc. corresponden a sensores que la estación no tiene.

Histórico · Cloud Function `obtener-datos-clima`

- **Transporte:** HTTPS con `CORS *`, sin autenticación. La URL se toma de una variable de entorno, porque cambia si la función se redespliega.
- **Parámetros:** `vista` y uno de estos filtros, aplicados en este orden de precedencia: `date_start[&date_end]`, `days`, `specific_date`.
- **Respuesta:** `{ total, data[] }`, ordenada de la más nueva a la más vieja, con **tope de 1000 filas** (unos 10 días), una fila cada 15 min y números como *string*.

- **Unidades:** métricas. Temperatura en °C, viento en km/h, lluvia en mm y mm/h, y presión en **mmHg**.

Vista	Columnas
`vw_clean_weather`	`id, fecha, hora, temp_out, hi_temp, low_temp, in_temp, in_hum_pct, out_hum_pct, rain, rain_rate, bar, wind_speed, hi_speed, hi_dir, wind_dir, temp_units, bar_units, wind_units, rain_units, bar_ajuste_mmhg`
`vw_temperatura`	`id, fecha, hora, temp_exterior, temp_max, temp_min, temp_interior, humedad_exterior, humedad_interior`
`vw_viento`	`id, fecha, hora, velocidad_kmh, rafaga_kmh, direccion, direccion_rafaga`
`vw_lluvia`	`id, fecha, hora, lluvia_mm, tasa_mm_hr`
`vw_presion`	`id, fecha, hora, presion_hpa, variacion_hpa, tendencia, ajuste_mmhg`

La app usa **`vw_clean_weather`**, que trae todo en una fila, para no hacer 4 consultas por tramo. Las demás vistas se documentan como referencia.

Rarezas y dónde se corrigen

Todas se corrigen en el mapper correspondiente de `packages/core`, y cada una tiene su test.

#	Rareza	Corrección	Estado
D1	Los arreglos `hl*` tienen 8 posiciones: [**mín. hoy, máx. hoy, hora mín., hora máx., máx. mes, mín. mes, máx. año, mín. año**]. Los dos mappers previos estaban mal: el web invertía máx. y mín., y los dos leían 4–7 como día y mes	`davis-live.mapper` usa el orden real (verificado con la lectura del 24-oct y con `hlbar`, cuyo mínimo del mes y del año es el valor sin corregir de antes del 20-oct)	Corregido en `@tt/core`
D2	`presion_hpa` y `bar` del histórico vienen en **mmHg**, aunque el nombre diga hPa	Convertir mmHg → hPa (× 1.33322)	Confirmado (`bar_units: "mm"`)
D3	**Presión corregida.** Desde el 2026-07-03, la calibración de la consola (`BARCAL`) bajaba `bar` 192.1 mmHg. Las vistas suman `ajuste_mmhg` y desde el 2026-10-20 la consola ya entrega el valor bueno. Hoy las dos fuentes dan unos 770 mmHg (unos 1027 hPa), un valor **reducido a nivel del mar**, comparable con METAR. **No es** la presión de estación (unos 578 mmHg a 2240 m), que mostraba la documentación vieja	La app etiqueta la presión como "a nivel del mar". Plausible: 740–790 mmHg; fuera de ese rango se trata como sospechosa	Confirmado: vivo `bar=30.430` inHg, histórico `bar=770.9`
D4	`bartr` es el **código Davis de tendencia**, no un delta: −60 baja rápido, −20 baja lento, 0 estable, 20 sube lento, 60 sube rápido	Mapear a `PressureTrend`	Confirmado: `bartr=60` con presión subiendo

#	Rareza	Corrección	Estado
D5	En las filas de la captura automática, `wind_dir` y `hi_dir` vienen **intercambiados**. Verificado el 5-nov: el CSV de la consola trae `...,Hi Speed,Hi Dir,Wind Dir, ...` y el `INSERT` de `weather-data-capture` (`main.py`) inserta esas columnas por posición como `hi_speed, wind_dir, hi_dir`, mientras que los `\copy` de julio–agosto usan `hi_speed, hi_dir, wind_dir`. Los datos no permiten distinguirlas (con ráfaga 0 las dos vienen vacías). Afecta la rosa de los vientos del histórico desde que corre la captura: muestra la dirección de la ráfaga, no la dominante	**Se corrige en el pipeline**, no en la app: cambiar el orden del `INSERT` y migrar las filas de la captura (hay que identificarlas por fecha: los imports de julio–agosto también llenan `source_file`). Al migrar, subir el prefijo de la caché del histórico (ADR-0013)	Confirmado; pendiente en el pipeline (fuera de la fase 8)
D6	`days=N` corta a medianoche UTC, no de Ciudad de México	No se usa `days`; siempre `date_start` y `date_end`	Decisión
D7	El histórico llega con **un día de retraso** (la captura corre a las 01:00)	Avisar en la UI si el rango incluye hoy (CU-04 A2)	Por diseño del pipeline
D8	Días perdidos para siempre: 2026-09-07, 09-15 y 09-16. Algunos días tienen menos de 96 filas	Los huecos se dibujan como cortes; nunca se interpolan	Por diseño
D9	`fecha` llega como ISO a medianoche UTC; la hora real está en `hora`	El instante se arma con `fecha[0..10] + hora` en America/Mexico_City	Decisión
D10	El dato en vivo trae `---` en sensores ausentes	Se convierte a `null`	Ya existe `parseValue`
D11	`date_start/date_end` aceptan `YYYY-MM-DD`, `YYYY-MM-DD HH:mm:ss` y `YYYY-MM-DDTHH:mm[:ss]`. Se comparan contra `fecha + hora` como hora de pared de CDMX, con **ambos extremos incluidos** (08:00–10:00 da 9 filas)	`HttpHistoryRepository` manda la hora de pared de CDMX y `Get History` quita el duplicado en la unión de tramos	Confirmado (la prueba anterior consultaba un día que aún no llegaba, por D7)
D12	El **reloj de pared** de la consola (`loctime`, horas de `hl*`) marca bien la hora de CDMX, pero su **zona horaria está configurada en UTC−5** (horario de verano abolido en 2022). Por eso `utctime` sale **1 h atrasado** y el amanecer y ocaso que calcula (`sunrt`, `sunst`) salen **1 h adelantados** (consola 7:35; real ≈ 06:35)	El instante sale de `loctime` como hora de CDMX; los `hl*` se leen como hora de CDMX; amanecer y ocaso se corrigen con el desfase que la consola cree tener (`loctime − utctime`), así que siguen bien si alguien arregla la consola	Confirmado el 24-oct comparando con el reloj del sistema. **Recomendado:** poner la consola en UTC−6 sin horario de verano

#	Rareza	Corrección	Estado
D13	Las horas del histórico vienen del reloj de pared de la consola, que es correcto (D12). Coincide con que la mínima del 19-oct sea a las 06:15, justo antes del amanecer real	Ninguna	Probable; confirmar en el pipeline

Escena del cielo

Viene de la nota `skyMap` del canvas de diseño. Las capas se evalúan **en orden** y una capa posterior puede reemplazar la escena que eligió una anterior.

Capa	Entrada	Regla	Escena
1. Hora del día	`sunrt`, `sunst`, hora actual	±50 min del amanecer → amanecer; ±50 min del ocaso → atardecer; entre ambos → día; si no → noche	base `clearDay` / `clearNight` / `dawn` / `twilight`
2. Cielo	texto de `forrule`	"Mostly clear" → despejado · "Partly cloudy" → `partly` · "Increasing clouds" → `increasing` · "Mostly cloudy" → `cloudy` (de noche `night Cloudy`) · "Clearing" o "Precipitation ending" → `clearing`	
3. Lluvia (manda la medición)	`rainr` (mm/h)	$0 < r < 0.5$ → `drizzle` · $r \geq 0.5$ → `rain` (de noche `nightRain`) · $r \geq 7.6$, o "possibly heavy at times" con lluvia medida → `storm` · sin lluvia medida pero con "Precipitation likely/continuing" → `likely`	
4. Viento	texto, ráfaga	"Windy" / "Increasing winds" o ráfaga ≥ 29 km/h → `windy` · "Possible wind shift to the W, NW, or N" → `wind Shift`	
5. Temperatura	texto	Solo con cielo despejado o parcial: "warmer" → `warmer`; "cooler" → `cooler`	
6. Neblina	humedad, temperatura, rocío, hora	humedad $\geq 90\,\%$ y (temperatura − rocío) < 2.5 °C, de noche o al amanecer y sin lluvia → `fog`	
7. Sistema	`forrule`, conexión	regla 193 → `calibrating`; código fuera de 0–194 o la consola no responde → `offline`	

Hay 21 escenas. Cada una tiene su artboard `project/Sky*.dc.html` y un caso de prueba que la produce (RF-APP-03).

Interpretación de variables (RF-APP-04)

Los umbrales que se usen en los textos de ayuda se documentan aquí antes de implementarlos:

Variable	Rangos para el texto	Fuente
Humedad relativa	$< 30\,\%$ seca · 30–60 % cómoda · 60–80 % húmeda · $> 80\,\%$ muy húmeda	Uso común (ASHRAE)
Diferencia temperatura − rocío	< 2.5 °C: probable neblina o rocío	Nota `skyMap`
Confort interior	20–24 °C y 40–60 % = cómodo	ASHRAE 55 (simplificado)
Beaufort	Tabla oficial 0–12 con su descripción	OMM
Tendencia de presión	Sube → mejora probable · baja → empeora probable · estable	Davis / convención

Implementación y desarrollo

Implementación

Este capítulo explica **cómo está construido** el código: qué hay en cada carpeta, cómo viaja un dato desde la estación hasta la pantalla y por qué las piezas están separadas así. Los capítulos 03 (arquitectura) y 04 (dominio) dicen *qué* forma tiene el sistema; aquí se recorre el código real con los nombres de archivo, para que alguien que llega nuevo sepa dónde buscar.

Tecnologías, en una frase cada una

Tecnología	Dónde	Qué aporta
TypeScript 6	En todo el repo	JavaScript con tipos: el compilador avisa si una función recibe datos con otra forma
React 19	Las dos apps	La UI se escribe como componentes (funciones que reciben datos y devuelven lo que se dibuja)
Next.js 16 (App Router)	`apps/web`	Framework de React para web: rutas por carpetas, servidor propio (para el proxy `/api/live`) y build de producción
Tailwind CSS 4	`apps/web`	Estilos con clases utilitarias (`rounded-md`, `text-wx-temp-text`) que leen los tokens del diseño
Expo SDK 58 + **React Native** 0.88	`apps/mobile`	React para Android/iOS: los componentes se convierten en vistas nativas. Expo da las librerías (archivos, compartir, fuentes) y la nube de builds y actualizaciones (EAS)
Expo Router	`apps/mobile`	Rutas por carpetas, como Next, pero para móvil (`src/app/(tabs)/...`)
react-native-svg y **reanimated** 4	`apps/mobile`	Dibujar SVG en móvil y animarlo en el hilo de UI
react-native-android-widget	`apps/mobile`	Widgets de la pantalla de inicio de Android escritos en React (ADR-0011)
Vitest, **jest-expo**, **Testing Library**, **MSW**, **axe-core**	Pruebas	Ver 08 · Calidad y pruebas
npm workspaces	Raíz	Un solo `npm install` y un solo `node_modules` para seis proyectos (ADR-0003)
GitHub Actions, **Cloud Run**, **EAS Update**	CI/CD	Ver 09 · Despliegue

Mapa del repositorio

.

```
├── apps/
│   ├── web/                 Next.js
│   │   ├── app/             rutas: page.tsx (/), historico/, api/live/route.ts, layout.tsx
│   │   ├── components/      now/, history/, shell/, providers/
│   │   ├── hooks/           use-live-weather.ts, use-history.ts
│   │   ├── lib/             core.ts (composition root), live/ (proxy), preferences/,
↪  download/, art/
│   │   ├── test/            Vitest + Testing Library + axe
│   │   └── Dockerfile       imagen standalone para Cloud Run (se construye desde la raíz)
│   └── mobile/              Expo
│       ├── index.ts         entrada: registra widgets (solo APK) y arranca Expo Router
│       ├── app.json, eas.json, metro.config.js
│       ├── src/app/         rutas: _layout.tsx, (tabs)/index.tsx (Ahora),
↪  (tabs)/historico.tsx
│       ├── src/components/  now/, history/, shell/, ui/, art/svg-art.tsx
│       ├── src/hooks/       los mismos dos hooks que la web
│       ├── src/lib/         core.ts, last-reading.ts, share-file.ts
│       ├── src/widget/      widget de Android
│       └── test/            jest-expo + Testing Library
├── packages/
│   ├── core/      @tt/core      dominio, casos de uso, puertos, adaptadores HTTP, geometría
↪  de gráficas
│   ├── i18n/      @tt/i18n      catálogos es/en, formateo en hora de CDMX, textos derivados
│   ├── tokens/    @tt/tokens    paleta clara/oscura, tipografía, hoja de variables CSS,
↪  contraste
│   └── sky-art/   @tt/sky-art   SVG de las 21 escenas y de la carga, sus animaciones (CSS y
↪  como datos)
├── docs/          esta guía
└── .github/       workflows de CI/CD y la acción release-tag
```

Cada carpeta importante tiene su propio `AGENTS.md` con las reglas locales (está pensado para asistentes de código, pero se lee igual de bien como guía rápida para personas).

La idea central: escribir la lógica una vez

Las dos apps muestran lo mismo ("Ahora" e "Histórico") con interfaces distintas. Antes de esta versión cada una tenía su propia lógica, con errores distintos (ver 00 · Visión). Ahora:

- **Todo lo que no es dibujar** vive en `packages/`: convertir unidades, decidir la escena del cielo, agrupar el histórico, calcular las escalas de las gráficas, armar los textos, exportar archivos.
- **Cada app solo dibuja** y aporta lo que depende de su plataforma: dónde se guardan las preferencias (`localStorage` o AsyncStorage), cómo se descarga un archivo (enlace del navegador u hoja de compartir), cómo se anima un SVG (CSS o reanimated).

Por eso, un componente típico de cualquiera de las dos apps se ve así: recibe un **modelo de vista** ya calculado por `@tt/core`, pide sus textos a `@tt/i18n`, toma sus colores de `@tt/tokens`, y no hace cuentas.

Arquitectura hexagonal en `@tt/core`

`packages/core/src` está partido en capas. La regla es que las flechas de importación solo apuntan hacia adentro:

Carpeta	Qué contiene	Puede importar
`domain/`	Reglas puras: unidades (`units.ts`), viento y Beaufort (`wind.ts`), presión (`pressure.ts`), sensación térmica y punto de rocío (`thermal.ts`), hora de CDMX (`time.ts`), sol (`sun.ts`), pronóstico Davis (`forecast.ts` y las 195 reglas de `forecast-rules.ts`), escena del cielo (`sky-scene.ts`), periodos del histórico	Solo `domain/`
`ports.ts`	Interfaces de lo que el núcleo necesita de afuera: `LiveWeather Repository`, `HistoryRepository`, `PreferencesRepository`, `Key ValueStorage`, `Clock`	`domain/`
`application/`	Casos de uso (`GetLiveWeather`, `GetHistory`, `ResolveCurrentSky`, `Get/UpdatePreferences`) y modelos de vista (`buildCurrent Conditions`, `buildHistoryView`, `liveStatus`, `exportHistory`, `widgetSummary`)	`domain/`, `ports.ts`, `charts/`
`charts/`	Geometría de gráficas sin dibujar nada: escalas "bonitas", trazos SVG (`d="M..."`), medidores, ilustraciones del histórico	`domain/`, modelos de vista
`adapters/`	Implementaciones de los puertos con `fetch` estándar: cliente HTTP con reintentos (`http.ts`), consola Davis (`davis-live/`), Cloud Function (`cloud-history/`), caché del histórico, preferencias sobre un almacén clave-valor	Todo lo anterior
`container.ts`	`createCore(config)`: arma todo y lo devuelve listo	Todo
`index.ts`	La API pública: lo único que importan las apps	Todo

El núcleo **no importa React, React Native, Next ni Expo**, ni librerías (salvo `hyparquet-writer`, aprobada por ADR-0012). No es una convención de palabra: `packages/core/test/architecture.test.ts` lee cada archivo y falla si alguien rompe la regla. Gracias a eso el mismo código corre en el navegador, en el servidor de Next, en Hermes (el motor de JS de React Native) y dentro del widget de Android.

Inyección de dependencias

Nada en el núcleo crea por su cuenta sus dependencias. Cada app tiene un *composition root* de pocas líneas:

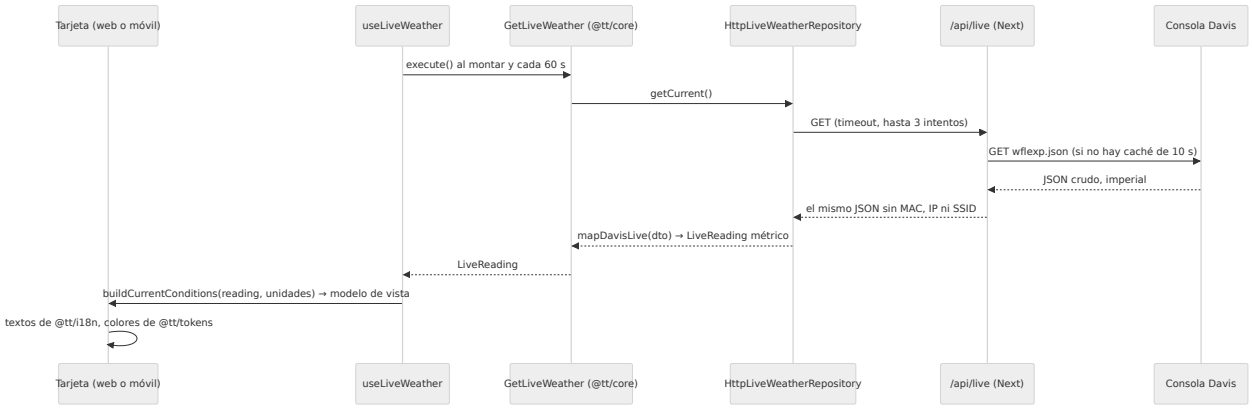
```
// apps/web/lib/core.ts
export function createWebCore(storage: Storage | undefined): Core {
  return createCore({ liveUrl: '/api/live', historyUrl:
  ↪  process.env.NEXT_PUBLIC_API_CLOUD_URL, storage })
}

// apps/mobile/src/lib/core.ts
export function createMobileCore(): Core {
  return createCore({
    liveUrl: process.env.EXPO_PUBLIC_WEATHER_STATION_URL,
    historyUrl: process.env.EXPO_PUBLIC_HISTORY_API_URL,
    storage: AsyncStorage,
  })
}
```

Un *provider* de React (`components/providers/app-providers.tsx` en web, `src/providers/app-providers.tsx` en móvil) guarda ese objeto y lo entrega con `useCore()`. En las pruebas, `AppProviders` recibe un `core` armado con repositorios en memoria y un reloj fijo, así que las pantallas se prueban sin red y con resultados reproducibles.

Recorrido 1 · El dato en vivo, de la estación a la tarjeta

Este es el camino más importante. Seguirlo con los archivos abiertos es la mejor forma de entender el proyecto.



1. **La consola.** La Davis Vantage Vue publica `wflexp.json` por HTTP en la red del IPN: valores en °F, mph e inHg, `---` cuando falta un sensor, y varias rarezas documentadas en 06 · Datos.
2. **El proxy** (`apps/web/app/api/live/route.ts` → `lib/live/live-proxy.ts`). El navegador no puede llamar a la consola (HTTP sin CORS desde una página HTTPS), así que el servidor de Next lo hace por él. Solo consulta la URL configurada, guarda la respuesta 10 s para no saturar la consola si hay muchos usuarios, y borra `mac`, `ip`, `ssid`. La app móvil usa el mismo proxy (ADR-0007).
3. **El cliente HTTP** (`packages/core/src/adapters/http.ts`). `fetch` con *timeout* y reintentos con espera creciente. Los errores salen tipados (`HttpError`), para que la UI distinga "no hay red" de "la consola falló".
4. **El mapper** (`adapters/davis-live/davis-live.mapper.ts`). Es el **único** lugar donde se corrigen las rarezas del dato en vivo: convierte a métrico, traduce `---` a `null` (nunca a 0), lee los extremos `hl*` en su orden real (D1), interpreta `bartr` como código de tendencia (D4) y corrige la zona horaria de la consola (D12). Sale una `LiveReading` (`domain/live-reading.ts`): un objeto limpio, en unidades canónicas, que ya no se parece al JSON de Davis.
5. **El caso de uso** `GetLiveWeather` solo pide la lectura al repositorio. Es pequeño a propósito: la lógica está en el dominio y la entrada/salida en los adaptadores.
6. **El hook** `useLiveWeather` (`apps/web/hooks/`, `apps/mobile/src/hooks/`). Consulta al montar, cada 60 s y cuando el usuario toca "actualizar". Cuenta los segundos desde la última consulta exitosa y le pide a `liveStatus` (`application/live-status.ts`) el estado: `loading`, `ok`, `stale` (más de 2 min), `offline` u `offlineWithData`. La vigencia se mide con el reloj del dispositivo, no con el de la consola, porque este deriva (D12).
7. **El modelo de vista** `buildCurrentConditions` (`application/current-conditions.ts`) hace todas las cuentas del panel en las unidades elegidas: sensación térmica y su motivo, barra del rango del día dentro del mes, punto de rocío, Beaufort, rumbo, intensidad de lluvia, avance del sol, escena del cielo, pronóstico. No produce texto.
8. **Los textos** salen de `@tt/i18n`: el catálogo (`catalogFor('es')`), el formateador de números y fechas en hora de CDMX (`createFormatter`) y los *presenters* (`valueText`, `windText`, `rainText`…), que combinan número, unidad y redacción igual en las dos apps.
9. **La tarjeta** (`components/now/*-card.tsx`) solo acomoda esos valores según el artboard del diseño.

La escena del cielo

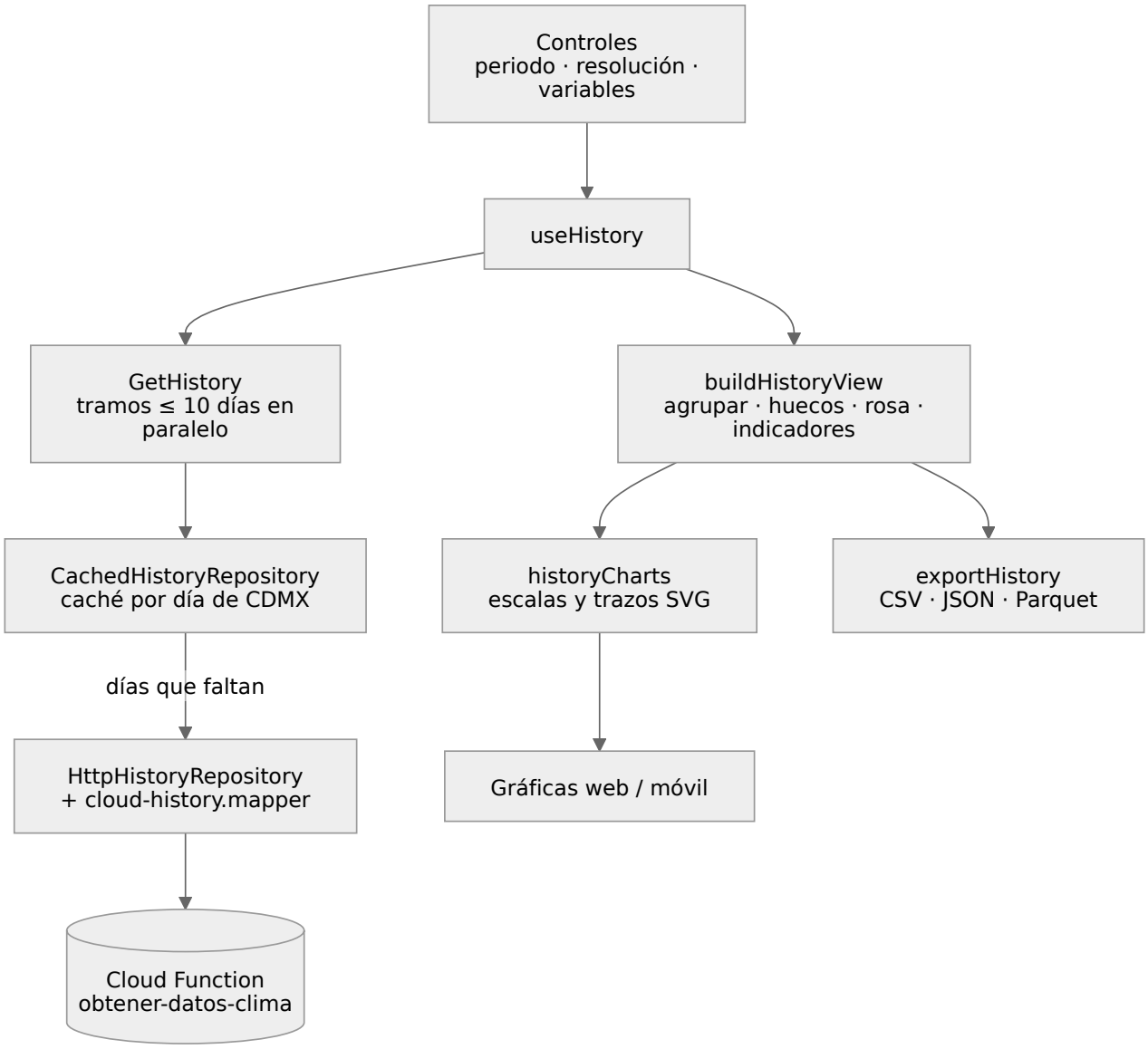
El héroe de "Ahora" muestra una de 21 ilustraciones animadas (despejado de día, llovizna, tormenta, niebla…).

- **Qué escena:** `resolveSkyScene` (`domain/sky-scene.ts`) evalúa siete capas en orden (sin datos → calibrando → pronóstico Davis → lluvia medida → niebla → viento → noche). La tabla completa está en 06

· Datos y cada regla tiene su caso en `test/domain/sky-scene.test.ts`.

- **El arte:** `@tt/sky-art` no se dibujó a mano. `npm run extract --workspace=@tt/sky-art -- <carpeta>` lee los artboards `Sky*.dc.html` del canvas de diseño y genera `scenes.generated.ts` (SVG) y `styles.generated.ts` (CSS), con prefijos para que varias escenas convivan sin choques (ADR-0009). Esos archivos no se editan a mano.
- **La animación en web** es el CSS del diseño tal cual. `lib/art/markup.ts` guarda objetos fijos para `dangerouslySetInnerHTML`: si se creara uno nuevo en cada render, React reescribiría el SVG y las animaciones volverían a empezar cada segundo (ese fue el bug del PR #10).
- **La animación en móvil:** React Native no entiende CSS, así que `motion.ts` describe las mismas animaciones como datos (pistas de fotogramas, duración, curva) y `apps/mobile/src/components/art/svg-art.tsx` las reproduce con reanimated en el hilo de UI (ADR-0010). `test/motion.test.ts` comprueba que los datos coinciden con las hojas CSS. Con "reducir movimiento" activado, en las dos plataformas el arte queda quieto.

Recorrido 2 · El histórico



1. **El periodo.** El usuario elige 24 h, 7 días, 30 días o "últimos N días". `periodRange` (`domain/history-period.ts`) lo traduce a un rango en hora de CDMX que **termina a la medianoche de hoy**, porque el pipeline carga los datos con un día de retraso (D7), y que no empieza antes del 19-abr-2026, el primer día con datos.
2. **Los tramos.** La Cloud Function corta en 1000 filas (≈ 10 días de lecturas cada 15 min). `GetHistory` parte el rango en tramos, los pide en paralelo y une los resultados quitando el duplicado de cada unión (la API incluye los dos extremos, D11).
3. **La caché** (`adapters/cached-history.repository.ts`, ADR-0013). Decora al repositorio HTTP: guarda cada día en el almacén clave-valor de la app (`wx-history-v1:AAAA-MM-DD`) y solo pide los días que no tiene. Los días que ya cerraron no cambian y se guardan sin vencimiento; los recientes vencen a la hora. Si el pipeline corrige datos viejos, se cambia el prefijo a `v2` y todas las cachés se invalidan solas.
4. **El mapper** (`adapters/cloud-history/cloud-history.mapper.ts`) corrige las rarezas del histórico: números como texto, filas de la más nueva a la más vieja, presión en mmHg con nombre de hPa (D2),

fecha y hora por separado (D9).

5. **La vista** `buildHistoryView` (`application/history-view.ts`) agrupa las lecturas por la resolución elegida (15 min, hora, día, semana lun–dom o mes, en hora de CDMX): promedio y banda mín–máx, lluvia sumada, máximo de ráfaga. Un intervalo sin lecturas queda vacío y se dibuja como corte, **nunca como cero** (D8). También calcula la rosa de los vientos y los indicadores (máxima, mínima, lluvia total, tendencia de presión).
6. **Las gráficas.** `historyCharts` (`charts/history-charts.ts`) convierte la vista en escalas con marcas redondas y en cadenas de trazo SVG (`d="M0,120L10,118..."`) sobre un lienzo de 1000 de ancho. Web y móvil solo ponen esas cadenas en un `<path>`; por eso no se usa ninguna librería de gráficas (ADR-0006) y las dos se ven igual.
7. **La descarga.** `exportHistory` produce exactamente lo que se ve (resolución y variables visibles) en CSV, JSON o Parquet (ADR-0012). La web lo entrega como descarga del navegador (`lib/download/save-file.ts`) y la móvil con la hoja de compartir del sistema (`src/lib/share-file.ts`).

Recorrido 3 · Preferencias, tema, idioma y tamaño de letra

- **El modelo:** `UserPreferences` (`domain/preferences.ts`): `theme` (sistema/claro/oscuro), `language` (es/en), `units` (métrico/imperial) y `textSize` (s/m/l). Un valor desconocido cae en el predeterminado, así que una clave corrupta no rompe la app.
- **Dónde se guardan:** `KeyValuePreferencesRepository` las escribe en las claves `wx-theme`, `wx-lang`… del almacén que le pase la app (`localStorage` o AsyncStorage). Son las mismas claves del diseño.
- **Tema en web:** `@tt/tokens` genera una hoja de variables CSS (`themeStylesheet()`) con la paleta clara en `:root` y la oscura en `:root.dark`. Tailwind expone esas variables como clases `bg-wx-*`, `text-wx-*`. Cambiar de tema es poner o quitar la clase `dark` en `<html>`; por eso **no se usan variantes** `dark:`. Un script en línea del `<head>` (`PREFERENCES_BOOT_SCRIPT` en `lib/preferences/apply.ts`) aplica tema, idioma y tamaño antes del primer pintado, para que no parpadee el tema equivocado.
- **Tema en móvil:** no hay CSS; el provider entrega el objeto `palette` del tema activo y cada componente toma sus colores de ahí. El texto pasa por el componente `Txt`, que pone la fuente IBM Plex, el color y la escala del tamaño elegido.
- **Contraste:** algunos acentos del diseño no llegaban a 4.5:1 como texto en tema claro. `@tt/tokens` agrega variantes `*-text` más oscuras para usar en texto, y `test/tokens.test.ts` calcula la razón WCAG de cada par.
- **Idioma:** los catálogos `catalogs/es.ts` y `catalogs/en.ts` son objetos con las mismas claves (el tipo `Catalog` sale del español y `test/catalogs.test.ts` verifica la paridad). Algunos textos son funciones (`stationName: (name) => ...`) para insertar valores.

Recorrido 4 · El widget de Android

- Existe solo en el APK (ADR-0011): la librería necesita su módulo nativo, que Expo Go no trae. Por eso `apps/mobile/index.ts` lo registra solo si `WIDGETS_AVAILABLE` y lo carga con un `require` diferido.
- Android llama a `src/widget/task-handler.ts` al agregar el widget, cada 30 min, al redimensionarlo o al tocar ↻. El manejador usa `widget-content.tsx`, que crea **el mismo núcleo** que la app (`createMobileCore`) y arma los textos con `buildCurrentConditions` y `widgetText` de `@tt/i18n`.
- Sin red (Android suele despertar el widget en modo Doze, sin conexión), dibuja la última lectura guardada en `wx-last-reading` (`src/lib/last-reading.ts`), en gris y con su hora. La app también guarda ahí cada lectura y se la pasa al widget, que así no la vuelve a pedir.
- `weather-widget.tsx` lleva `'use no memo'`: el React Compiler transformaba esos componentes y el widget fallaba con "Invalid Hook Call".

Detalles de la plataforma que conviene conocer

- **Next sustituye `process.env.NEXT_PUBLIC_*` al compilar.** Por eso `route.ts` lee cada variable con su nombre literal: en Cloud Run esas URLs solo llegan como argumento del build.
- **Metro y `hyparquet-writer`.** Esa librería publica una entrada para Node (que importa `fs`) y otra para navegador. `apps/mobile/metro.config.js` hace que Metro use la de navegador solo para ella.
- **El Dockerfile de la web se construye desde la raíz** y copia `packages/`: si se agrega un paquete compartido nuevo, también hay que agregar su `package.json` al `Dockerfile`.
- **`/android` y `/ios` no se versionan.** El proyecto nativo se genera en cada build a partir de `app.json` (*Continuous Native Generation*). Cualquier ajuste nativo se hace con *config plugins* en `app.json`, no editando `/android`.
- **Expo Router 58** cambió la API de `tabBar` (recibe `emitter` y `navigateToTab`). Antes de tocar APIs de Expo, consulta la documentación de la versión exacta: https://docs.expo.dev/versions/v58.0.0/.
- **Los hooks están copiados** en web y móvil (`useLiveWeather`, `useHistory`), porque son React y los paquetes no usan React. Si crecen, la salida es un paquete `@tt/react` con hooks que reciban el núcleo como parámetro.

Cómo encontrar algo

Quiero cambiar…	Empieza en
Un cálculo (sensación térmica, Beaufort, tendencia de presión)	`packages/core/src/domain/` y su test en `packages/core/test/domain/`
Cómo se interpreta un campo de la consola o del histórico	El mapper en `packages/core/src/adapters/*/` y 06 · Datos
Qué escena del cielo sale	`packages/core/src/domain/sky-scene.ts`
Un texto o su traducción	`packages/i18n/src/catalogs/es.ts` y `en.ts`
Cómo se redacta un valor con su unidad	`packages/i18n/src/presenters.ts`
Un color	`packages/tokens/src/palette.ts`
Cómo se ve una tarjeta	`apps/web/components/now/` o `apps/mobile/src/components/now/`
Las gráficas del histórico	`packages/core/src/charts/` (geometría) y `components/history/` de cada app (dibujo)
La descarga	`packages/core/src/application/export-history.ts`
El proxy	`apps/web/lib/live/live-proxy.ts`
El widget	`apps/mobile/src/widget/`
CI/CD	`.github/workflows/` y 09 · Despliegue

Entorno de desarrollo

Cómo dejar una computadora lista para correr, probar y modificar las dos apps. No hace falta experiencia previa con Expo ni con Next.js: cada paso dice qué hace y cómo saber que salió bien.

Lo que se necesita

Herramienta	Versión	Para qué	¿Obligatoria?
Git	cualquiera reciente	Clonar el repo y trabajar en ramas	Sí
Node.js	22 o más reciente (`engines` del `package.json` raíz)	Correr todo: apps, tests, scripts	Sí
npm	el que trae Node	Instalar dependencias (el repo usa *npm workspaces*, ADR-0003)	Sí
Editor	VS Code recomendado (`.vscode/extensions.json` sugiere extensiones)	Editar con TypeScript y autocompletado	Recomendado
Expo Go	la que corresponde al **SDK 58** (tiendas de Android/iOS)	Abrir la app móvil en un teléfono sin compilarla	Para móvil
JDK 17 + Android SDK	(por ejemplo, con Android Studio)	Compilar el APK en local	Solo para el APK
EAS CLI	`npm i -g eas-cli`	Compilar el APK, publicar actualizaciones, ver variables de EAS	Solo para el APK y la operación
`gh` (GitHub CLI)	cualquiera reciente	Abrir PR y revisar el CI desde la terminal	Opcional

No hace falta instalar pnpm, yarn, Docker ni Java para desarrollar: solo para el APK (Java) o para probar la imagen de producción de la web (Docker).

Primeros pasos

```
git clone https://github.com/Samu31Nd/weather_app.git
cd weather_app
npm install            # en la raíz: instala las dos apps y los cuatro paquetes de una vez
```

`npm install` siempre se corre **en la raíz**, nunca dentro de `apps/web` o `apps/mobile`. Los *workspaces* enlazan los paquetes `@tt/*` dentro de `node_modules/@tt/`, así que las apps los importan como si fueran librerías publicadas, pero leen el código fuente directo (no hay paso de compilación de los paquetes).

Variables de entorno

Cada app lee su propio `.env`. Las plantillas están versionadas y los valores reales **nunca** se commitean:

```
cp apps/web/.env.template apps/web/.env
cp apps/mobile/.env.example apps/mobile/.env
```

App	Variable	Valor
web	`NEXT_PUBLIC_API_URL`	URL base de la consola Davis (termina en `/`; el proxy le agrega `wflexp.json`). Alternativa: `LIVE_STATION_URL` con la URL completa de `wflexp.json`
web	`NEXT_PUBLIC_API_CLOUD_URL`	URL de la Cloud Function `obtener-datos-clima`
móvil	`EXPO_PUBLIC_WEATHER_STATION_URL`	El proxy de la web: `https://<servicio>.run.app/api/live`. En desarrollo también puede ser la consola directa (`http://<ip>/wflexp.json`), porque Expo Go permite HTTP
móvil	`EXPO_PUBLIC_HISTORY_API_URL`	La misma Cloud Function de la web

Las URLs vigentes las tiene quien opera el sistema; las de producción también están en las variables del repositorio de GitHub (*Settings* → *Secrets and variables* → *Actions*) y en las variables de entorno de EAS (`eas env:list --environment production`). Todas son URLs públicas, no secretos. Lo que **sí** es secreto (`EXPO_TOKEN`, el estado de Terraform de `proj-files`) no se copia nunca al repo.

> **Sin acceso a la consola.** La consola Davis está en la red del IPN. Desde fuera, apunta la app móvil al proxy de producción y la web a una copia de `wflexp.json` servida localmente. Por ejemplo, con `python3 -m http.server 8080 -d packages/core/test/fixtures` y `LIVE_STATION_URL=http://localhost:8080/wflexp-2026-10-24.json` en `apps/web/.env`, "Ahora" muestra esa lectura real del 24-oct. El histórico sí es accesible desde cualquier red.

Correr las apps

```
npm run web       # Next.js en http://localhost:3000 (recarga al guardar)
npm run mobile    # Expo: muestra un QR
```

- **Web.** Abre http://localhost:3000. Debe aparecer la pantalla de carga y luego "Ahora". Si se queda en "sin conexión", revisa `NEXT_PUBLIC_API_URL`: http://localhost:3000/api/live debe responder JSON.
- **Móvil.** Escanea el QR con Expo Go (Android) o con la cámara (iOS); el teléfono y la computadora deben estar en la misma red. Si no lo están, `npx expo start --tunnel` desde `apps/mobile`. Las variables `EXPO_PUBLIC_*` se leen al iniciar el servidor: después de cambiar `.env`, reinicia con `npx expo start -c` (borra la caché de Metro).
- **Widget de Android.** No existe en Expo Go (es código nativo, ADR-0011). Para verlo hay que instalar un APK.

Verificar antes de subir cambios

Son los mismos pasos que corre el CI (`.github/workflows/ci.yml`):

```
npm run typecheck                 # TypeScript en los seis workspaces
npm test                          # todas las pruebas (core con umbral de cobertura)
npm run build --workspace=web     # compila la web de producción
```

Comandos útiles para trabajar en una parte:

```
npm test --workspace=@tt/core              # o @tt/i18n, @tt/tokens, @tt/sky-art, web,
↪ mobile
npx vitest --root packages/core            # modo watch: corre de nuevo al guardar
npx vitest run --root packages/core test/domain/wind.test.ts   # un solo archivo
npm test --workspace=mobile -- now-screen  # jest: filtra por nombre de archivo
npm run test:contract --workspace=@tt/core # contra las fuentes reales (necesita red y
↪ URLs)
```

Compilar el APK

Solo hace falta para probar el widget o una dependencia nativa; los demás cambios llegan a la app instalada por EAS Update (ver 09 · Despliegue).

```
mkdir -p ~/tmp-build
eas login                                  # cuenta con acceso al proyecto
↪ @samu31nds-team/tt-b033
TMPDIR=~/tmp-build npm run build:android   # eas build --local; deja el .apk en
↪ apps/mobile/
adb install -r apps/mobile/*.apk
```

`TMPDIR` debe estar en disco: Gradle usa unos 3.5 GB y, si `/tmp` está en RAM, falla con *No space left on device*. Las URLs del APK salen de las variables de entorno de EAS, no de `.env`.

Problemas comunes

Síntoma	Causa probable	Solución
`Cannot find module '@tt/core'`	`npm install` se corrió dentro de una app	Borra `node_modules` de esa app y corre `npm install` en la raíz
TypeScript no encuentra `className` en componentes de RN, o hay dos copias de reanimated	`package-lock.json` arrastra versiones viejas tras cambiar de SDK de Expo	Regenera el lock desde cero (ver 14 · Mantenimiento)
Expo Go dice que el proyecto es incompatible	Expo Go solo abre el SDK más reciente	Actualiza Expo Go; si el SDK nuevo ya salió, toca migrar el proyecto
La móvil no ve un cambio de `.env`	Metro cachea las `EXPO_PUBLIC_*`	`npx expo start -c`
"Ahora" muestra "sin conexión" en local	La consola no es accesible desde tu red	Usa el proxy de producción o un `wflexp.json` local (arriba)
El histórico de hoy sale vacío	El pipeline carga los datos con un día de retraso (D7)	Es lo esperado: el periodo termina a la medianoche de hoy
El APK abre sin datos	El APK no lee `.env`	Revisa `eas env:list --environment production`
`npx expo lint` creó `eslint.config.js`	Ese comando instala ESLint si no existe	Descarta los archivos nuevos si no querías agregar ESLint

Cómo contribuir

Esta guía es para quien quiera corregir un error, agregar una función o continuar el proyecto: estudiantes que lo retomen como trabajo terminal, servicio social o práctica, y profesores que quieran proponer cambios. No se necesita permiso para leer ni para experimentar en una copia; para que un cambio llegue a producción hay que seguir el flujo de abajo.

Antes de empezar

1. Deja listo el entorno (11 · Entorno de desarrollo) y comprueba que `npm run typecheck` y `npm test` pasan **antes** de tocar nada. Así sabes que lo que falle después es tuyo.
2. Lee lo que vas a tocar: el requerimiento (`RF-APP-NN` en 01 · Requerimientos), su caso de uso (02), y el capítulo de implementación de esa parte.
3. Si el cambio es grande o cambia una decisión, revisa los ADR (adr/). Si vas a contradecir uno, escribe uno nuevo que lo reemplace (ver abajo).

El flujo, paso a paso



1. **Rama.** Siempre desde `main` actualizado. Nombres: `feat/...` (función nueva), `fix/...` (error), `refactor/...`, `docs/...`. En minúsculas y con guiones: `fix/rain-card-units`.
2. **Commits.** Formato Conventional Commits: `<tipo>(<alcance>): <descripción>`, en imperativo y minúscula, sin punto final. Título corto y, si hace falta, un cuerpo de 1–3 líneas.
   - Tipos: `feat`, `fix`, `refactor`, `perf`, `style`, `test`, `docs`, `build`, `ops`, `chore`.
   - Alcances usados: `core`, `web`, `mobile`, `i18n`, `tokens`, `sky-art`, `ci`.
   - Ejemplos reales del historial: `fix(mobile): reset dashed border on history chips`, `docs: record the apk accessibility review with uiautomator`.
   - Un cambio incompatible lleva `!` (`feat(core)!: ...`) o un pie `BREAKING CHANGE:`.
3. **Pull request** contra `main`. La descripción dice qué cambia, por qué, qué IDs toca (`RF-APP-07`, `CU-04`, `D5`…) y cómo se probó. Si cambia la UI, una captura ayuda.
4. **CI.** `ci.yml` corre solo los jobs de lo que cambió (`packages`, `web`, `mobile`) y los resume en `ci-ok`, el único check requerido. Sin `ci-ok` en verde no se puede mergear, ni siquiera siendo administrador.
5. **Staging (opcional).** Un colaborador del repo comenta en el PR exactamente `cicd/deploy-staging-web` o `cicd/deploy-staging-app` y el bot responde con la URL o el QR de una vista previa. Se borra al cerrar el PR.
6. **Merge** con *squash*. Al llegar a `main`, `release.yml` publica en producción solo la app afectada (web en Cloud Run, móvil por EAS Update) y crea el tag `web-v...` o `mobile-v...`.

   Sin acceso de escritura (por ejemplo, si el repo ya pasó a otra cuenta): haz *fork*, trabaja en tu copia y abre el PR desde ahí. El CI corre igual; el staging y la publicación los dispara alguien con

acceso.

Lista de revisión de un PR

Quien abre el PR la revisa antes de pedir revisión, y quien revisa la usa como guía:

- ☐ `npm run typecheck` y `npm test` pasan en local.
- ☐ La lógica nueva está en `packages/`, no repetida en las dos apps.
- ☐ `@tt/core` no importa nada de React, Next, Expo ni librerías (`architecture.test.ts` lo vigila).
- ☐ Una rareza de datos se corrige **en el mapper** y tiene su test con el ID `Dn` de 06 · Datos.
- ☐ "Sin dato" sigue siendo `null`, no 0.
- ☐ Los textos nuevos están en `es.ts` y en `en.ts`.
- ☐ Los colores salen de `@tt/tokens`; para texto de color, las variantes `*-text`.
- ☐ Cada control nuevo tiene rol, nombre accesible y área táctil de al menos 44 dp (móvil).
- ☐ Los tests nuevos tienen cabecera (qué regla, de dónde sale, qué RF) y `describe`/`it` en español.
- ☐ Si el cambio contradice algo de `docs/`, `docs/` se actualiza en el mismo PR.
- ☐ No se commitean `.env`, `*.apk`, tokens ni archivos de `proj-files` (`terraform.tfvars`, `*.tfstate`).

Convenciones de código

- **Nombres que expliquen la intención**, en inglés en el código (`resolveSkyScene`, `isPlausibleSeaLevelPressure`); comentarios, documentación y tests en español.
- **Funciones cortas y puras** donde se pueda: entrada → salida, sin leer el reloj ni la red por su cuenta (el reloj se inyecta con `Clock`).
- **Constantes con nombre** para los umbrales (`DRIZZLE_MAX_MM_H = 0.5`, `STALE_AFTER_MS`), exportadas para que el test las use.
- **Cada archivo empieza con un comentario** que dice qué es y de dónde salen sus reglas (requerimiento, nota del diseño, rareza de datos).
- **Sin código muerto** ni bloques comentados: el historial de git ya los guarda.
- **Tipos estrictos.** `readonly` en los modelos, uniones de literales (`'metric' | 'imperial'`) en vez de strings libres, y nada de `any`.
- **Estilos en web:** utilidades `bg-wx-*`/`text-wx-*`, sin variantes `dark:`; cifras con la clase `num`.
- **Estilos en móvil:** colores de `palette`, texto con `Txt`, sin `fontWeight` (cada peso es otra familia).
- Formato: el repo no tiene ESLint ni Prettier. Sigue el estilo del archivo que editas (dos espacios, comillas simples, sin punto y coma, líneas de hasta ~120 caracteres).

Cómo escribir pruebas

Toda regla nueva lleva su prueba en el mismo PR. La guía completa está en 08 · Calidad y pruebas; en corto:

```
/**
 * Viento - escala de Beaufort a partir de km/h.
 * Fuente: tabla de la OMM; docs/06-datos.md#interpretación-de-variables.
 * Requerimientos: RF-APP-01, RF-APP-04.
 */
describe('beaufortFromKmh', () => {
  it.each([
    { caso: 'calma', kmh: 0.5, fuerza: 0 },
    { caso: 'límite inferior de Beaufort 4', kmh: 20, fuerza: 4 },
  ])('$caso: $kmh km/h es fuerza $fuerza', ({ kmh, fuerza }) => {
```

```
    expect(beaufortFromKmh(kmh)).toBe(fuerza)
  })
})
```

- **Dónde:** el test vive en la carpeta `test/` del paquete o app, con la misma ruta que el archivo que prueba (`src/domain/wind.ts` → `test/domain/wind.test.ts`).
- **Datos reales:** para mappers, usa los fixtures de `packages/core/test/fixtures/` (con su README). Si una fuente cambia de forma, captura un fixture nuevo con fecha y conserva el anterior.
- **Pantallas:** monta la pantalla con `AppProviders` y un `core` armado con dobles (repositorios en memoria, reloj fijo). Busca por texto o rol accesible, como lo haría una persona (`getByRole('button', { name: 'Actualizar' })`).
- **Cobertura:** `@tt/core` exige al menos 80 % de líneas; el CI falla por debajo.
- **Trazabilidad:** si el test cubre un RF nuevo, agrégalo a la matriz de 08.

Recetas

Agregar o cambiar un texto

1. Agrega la clave en `packages/i18n/src/catalogs/es.ts` (el tipo `Catalog` sale de este archivo).
2. Agrega la misma clave en `en.ts`; TypeScript marca el error si falta, y `catalogs.test.ts` también.
3. Úsala en el componente con `t.seccion.clave` (el catálogo llega por `usePreferences()`).

Mostrar un dato nuevo de la consola

Supón que quieres mostrar la radiación solar, si la consola la publicara.

1. **Datos.** Documenta el campo en `apps/web/docs/realtime-data-schema.md` y en 06 · Datos, con su unidad y sus rarezas.
2. **DTO.** Agrega el campo a `adapters/davis-live/davis-live.dto.ts` (tal como llega: `string | null`).
3. **Dominio.** Agrega la propiedad, ya en unidades canónicas, a `LiveReading` (`domain/live-reading.ts`).
4. **Mapper.** Conviértelo en `davis-live.mapper.ts` con `parseNumber` (que ya traduce `---` a `null`), y pruébalo con un fixture real en `test/adapters/davis-live.mapper.test.ts`.
5. **Modelo de vista.** Si hay cálculo o conversión de unidades, va en `buildCurrentConditions`, con su test.
6. **Textos** en `@tt/i18n` (catálogos y, si combina número y unidad, un *presenter*).
7. **UI.** Una tarjeta en `apps/web/components/now/` y otra en `apps/mobile/src/components/now/`, cada una según su artboard. Prueba de pantalla en cada app.
8. **Serialización.** Si el widget debe ver el dato sin red, revisa `adapters/live-reading-json.ts`.
9. **Docs.** Requerimiento o criterio en 01 si hace falta, y la fila de la matriz en 08.

Corregir una rareza nueva de los datos

1. Reprodúcela con un dato real y guárdalo como fixture (anonimizado: sin MAC, IP ni SSID).
2. Dale el siguiente ID (`D14…`) en la tabla de 06 · Datos: qué pasa, cómo se corrige y cómo se confirmó.
3. Corrígela **solo** en el mapper correspondiente, con un test que cite el ID.
4. Si cambia datos del histórico que ya están en caché, sube el prefijo `HISTORY_CACHE_PREFIX` a la siguiente versión (`wx-history-v2:`), como dice ADR-0013.
5. Si el error está en el pipeline (fuera de este repo), anótalo como pendiente en lugar de parcharlo en la app.

Agregar o cambiar una escena del cielo

1. La regla va en `domain/sky-scene.ts` (y el ID nuevo en `SKY_SCENES`), con su caso en `test/domain/sky-scene.test.ts`.

2. El arte se dibuja en el canvas de diseño (artboard `Sky<Nombre>`) y se extrae con `npm run extract -- workspace=@tt/sky-art -- <carpeta con los Sky*.dc.html>`. No edites `*.generated.ts` a mano.
3. Si trae animaciones nuevas, agrégalas como datos en `packages/sky-art/src/motion.ts`; `motion.test.ts` revisa que coincidan con el CSS.
4. Nombre de la escena en `es.ts` y `en.ts`, y actualiza la tabla de 06 · Datos.

Agregar una dependencia

- **En `@tt/core`, casi nunca.** Solo librerías de JS puro sin APIs de plataforma, con un ADR que lo justifique y agregándola a la lista de `test/architecture.test.ts` (así entró `hyparquet-writer`, ADR-0012).
- **En la app móvil**, instala con `npx expo install <paquete>` desde `apps/mobile` para que Expo elija la versión compatible con el SDK. Si el paquete tiene código nativo, solo funcionará en un APK nuevo, no por EAS Update.
- **Herramientas de prueba** (vitest, vite, msw) van en el `package.json` de la raíz, no en el paquete: npm no resuelve bien el *hoisting* de vite si quedan dentro.
- Después de instalar, confirma que `package-lock.json` cambió y súbelo en el mismo commit.

Registrar una decisión (ADR)

Cuando una decisión afecta la arquitectura o es difícil de revertir (una librería nueva, otro hosting, cambiar cómo se guarda algo), escribe `docs/adr/NNNN-titulo-corto.md` con el siguiente número y estas secciones:

```
# ADR-0014 · Título en forma de decisión

- **Estado:** Aceptada (AAAA-MM-DD)

## Contexto
Qué problema había, qué restricciones aplicaban y qué alternativas se consideraron.

## Decisión
Qué se eligió, en concreto (archivos, librerías, reglas).

## Consecuencias
Qué se gana, qué se pierde y qué hay que vigilar.
```

Agrégalo a la tabla de adr/README.md. Un ADR no se borra: si la decisión cambia, el nuevo dice a cuál reemplaza y el viejo se marca **Reemplazada por ADR-NNNN**.

Actualizar la documentación y el PDF

- La documentación es parte del cambio: si el código contradice `docs/`, se corrige `docs/` en el mismo PR.
- Los IDs (`RF-APP-NN`, `CU-NN`, `Dn`, ADR) nunca se reutilizan; lo retirado se marca **Retirado**.
- Los diagramas se escriben en Mermaid dentro del Markdown (GitHub los dibuja).
- El PDF de esta guía se regenera con `docs/pdf/build.sh` (ver README del PDF).

Calidad y pruebas

Estrategia por capa

Capa	Herramienta	Qué se prueba	Dobles
Dominio (`core/domain`)	Vitest	Reglas puras (y `test/architecture.test.ts`: el núcleo solo importa archivos propios): conversiones, Beaufort, rumbos, sensación térmica, decodificación del pronóstico, escena del cielo, KPIs, remuestreo, geometría de gráficas	Ninguno: entradas y salidas
Aplicación (`core/application`)	Vitest	Orquestación: partir rangos, unir y quitar duplicados, validar preferencias	Repositorios en memoria, `Clock` fijo
Adaptadores (`core/adapters`)	Vitest + MSW	Integración HTTP: mapeo de fixtures reales, `---`, errores, timeouts, reintentos	MSW sirve los fixtures de `packages/core/test/fixtures/`
Contrato	Vitest (`npm run test:contract`)	Que las fuentes reales siguen dando la forma esperada. No corre en cada PR	Ninguno: endpoints reales. Se activa con `CONTRACT=1`
Web	Vitest + Testing Library	Componentes clave (tarjetas, estados, ajustes) y el route handler del proxy	`createContainer` con dobles; MSW para el proxy
Web e2e	Playwright	Smoke: carga, cambio de tema, cambio de idioma, navegación, descarga	Fixtures servidos por el proxy en modo de prueba
Móvil	jest-expo + RN Testing Library	Hooks y pantallas: estados, preferencias, tamaño de texto	`createContainer` con dobles

Cómo se documenta una prueba

Cada archivo empieza con un bloque que dice qué cubre, de dónde sale la regla y qué requerimientos traza:

```
/**
 * Escena del cielo - capa 3 (lluvia medida manda sobre el pronóstico).
 * Fuente: nota `skyMap` del canvas de diseño; docs/06-datos.md#escena-del-cielo.
 * Requerimientos: RF-APP-03.
 */
describe('ResolveSkyScene · lluvia', () => {
  it('elige llovizna cuando 0 < rainr < 0.5 mm/h aunque el pronóstico diga despejado', ()
  ↪  => { ... })
})
```

- Los `describe` e `it` van en español y se redactan como comportamiento observable.
- Las tablas (`it.each`) llevan una columna `caso` que explica por qué existe cada fila. Por ejemplo: "límite inferior de Beaufort 4".
- Los fixtures tienen un `README.md` que dice de dónde salieron (endpoint y fecha) y que confirma que no traen datos internos: la MAC, la IP y el SSID van anonimizados.

Cómo correrlas

```
npm test --workspace=@tt/core          # unitarias + integración, con cobertura
npm run test:contract --workspace=@tt/core   # contra las fuentes reales (no corre en el
↪ CI)
npx vitest --root packages/core        # modo watch
npm test --workspace=web               # Vitest + Testing Library (jsdom)
npm test --workspace=mobile            # jest-expo + RN Testing Library
```

En móvil, `test/setup.ts` sustituye reanimated por una versión estática: el mock oficial de reanimated 4 carga el módulo nativo y falla en jest. Las animaciones se prueban como datos en `packages/sky-art/test/motion.test.ts`.

Umbrales

- Cobertura de líneas de `packages/core` $\geq 80\,\%$ (RNF-APP-09). El CI falla si baja de ahí.
- En las apps no se exige cobertura. Se exige que **cada RF** tenga al menos una prueba en la matriz.

Matriz de trazabilidad

Se llena conforme avanzan las fases. "Componente" es el módulo que implementa el requerimiento y "Prueba" es el archivo de test.

RF / RNF	CU	Componente	Prueba	Fase
RF-APP-01	CU-01	mapper de `core/adapters/davis-live` · reglas de `core/domain` (sensación, Beaufort, rumbos, sol, confort) · textos de `i18n/presenters` · tarjetas de "Ahora"	`packages/core/test/adapters/davis-live.mapper.test.ts`, `domain/{units,wind,thermal,time-and-sun}.test.ts`; `packages/i18n/test/presenters.test.ts`; `apps/mobile/test/now-screen.test.tsx`	1, 3, 4
RF-APP-02	CU-01, 02	`useLiveWeather` y máquina de estados de "Ahora" (web y móvil) · pantallas de carga (`loadingProgress`)	`apps/web/test/now-screen.test.tsx`, `apps/mobile/test/now-screen.test.tsx` (carga → panel, reintento); `packages/core/test/application/{live-status,loading-progress}.test.ts`; `packages/i18n/test/widget-text.test.ts`, `apps/mobile/test/{weather-widget,widget-content}.test.tsx` (widget: datos, última lectura guardada, sin conexión, ⟳); `packages/core/test/adapters/live-reading-json.test.ts`	3, 4, 7

RF / RNF	CU	Componente	Prueba	Fase
RF-APP-03	CU-08	`core/domain/sky-scene` · `core/domain/forecast` · arte y animaciones de `@tt/sky-art` · escena web y móvil (`SvgArt`)	`packages/core/test/domain/sky-scene.test.ts` (21 escenas), `domain/forecast.test.ts`, `application/use-cases.test.ts`; `packages/sky-art/test/{sky-art,motion}.test.ts`; `apps/mobile/test/svg-art.test.tsx`; `packages/core/test/application/widget-summary.test.ts`, `packages/sky-art/test/widget-icons.test.ts` (widget)	1, 3, 4, 7
RF-APP-04	CU-03	`core/domain/interpretation` · textos de ayuda en `packages/i18n`	`packages/core/test/domain/thermal.test.ts` (bandas y confort; los textos llegan en la fase 2)	1, 2
RF-APP-05	CU-07	mapper (descarta MAC/IP/SSID) · tarjeta de estado	`packages/core/test/adapters/davis-live.mapper.test.ts` (sin MAC/IP/SSID)	1, 3, 4
RF-APP-06	CU-04	`GetHistory` (tramos, unión, orden) · `CachedHistoryRepository` (caché por día, vigencia, tope) · periodos en hora de CDMX (`history-period`) · controles de periodo web y móvil	`packages/core/test/domain/{history,history-period}.test.ts`, `application/use-cases.test.ts`, `adapters/{http,cached-history.repository}.test.ts`; `apps/web/test/history-screen.test.tsx` (90 días en 9 tramos)	1, 5
RF-APP-07	CU-04	`buildHistoryView` (agregación, huecos) · `charts/{series,history-charts}` · gráficas web y móvil	`packages/core/test/application/history-view.test.ts`, `charts/{series,history-charts}.test.ts`; `packages/i18n/test/history-text.test.ts`; `apps/web/test/history-screen.test.tsx`, `apps/mobile/test/history-screen.test.tsx`	5
RF-APP-08	CU-04	indicadores de `buildHistoryView` · `historyArt` · `historySummaryText`	`packages/core/test/application/history-view.test.ts` (valores calculados a mano y fixture real), `charts/history-art.test.ts`; `packages/i18n/test/history-text.test.ts`	5
RF-APP-09	CU-05	`exportHistory` (CSV, JSON, Parquet) · descarga web (`saveFile`) y hoja de compartir móvil (`shareFile`)	`packages/core/test/application/export-history.test.ts` (Parquet leído de vuelta con `hyparquet`); `apps/web/test/history-screen.test.tsx`, `apps/mobile/test/history-screen.test.tsx`	5
RF-APP-10	CU-06	`UserPreferences` · adaptadores de almacenamiento · providers de tema, idioma y tamaño · hoja de ajustes móvil	`packages/core/test/domain/preferences.test.ts`, `adapters/key-value-preferences.test.ts`; `apps/web/test/preferences-dom.test.ts`, `now-screen.test.tsx` (idioma, unidades, tema); `apps/mobile/test/now-screen.test.tsx` (ajustes y claves `wx-*` en AsyncStorage)	2, 4

RF / RNF	CU	Componente	Prueba	Fase
RNF-APP-01	CU-01, 04	mappers de `core/adapters`	`packages/core/test/adapters/*.mapper.test.ts` con fixtures reales; `test/contract/` contra las fuentes	1
RNF-APP-02	CU-01, 04	adaptadores HTTP (reintentos) · estados de pantalla	`packages/core/test/adapters/http.test.ts` (reintentos, timeout, 4xx/5xx, tope de filas)	1, 3, 4
RNF-APP-05	—	todas las UI	`packages/tokens/test/{tokens, widget}.test.ts` (contraste AA en ambos temas, también en el widget), `packages/sky-art/test` (movimiento reducido en web; en móvil `SvgArt` usa `useReducedMotion`); `apps/web/test/accessibility.test.tsx` (axe-core, WCAG 2.2 A/AA, en carga, "Ahora" e "Histórico"); `apps/mobile/test/accessibility.test.tsx` (rol, nombre y área táctil ≥ 44 dp de cada control); revisión manual con TalkBack (abajo)	2–5, 8
RNF-APP-07	—	`packages/i18n`	`packages/i18n/test/catalogs.test.ts` (paridad es/en), `format.test.ts`, `forecast-text.test.ts`	2
RNF-APP-08	—	`packages/core`	`packages/core/test/architecture.test.ts` (regla de dependencias)	1
RNF-APP-09	—	`packages/core`	umbral de cobertura en `packages/core/vitest.config.ts` (job `core` del CI)	0b, 1
RNF-APP-10	CU-01	`/api/live`	`apps/web/test/live-proxy.test.ts` (URL fija, sin MAC/IP/SSID); `now-screen.test.tsx`	3
RNF-APP-11	CU-01	`/api/live` (caché) · caché de sesión del histórico	`apps/web/test/live-proxy.test.ts` (caché de 10 s)	3, 5

Accesibilidad (RNF-APP-05)

La parte automática corre en el CI con los demás tests:

- **Contraste:** `packages/tokens/test/{tokens,widget}.test.ts` calcula la razón WCAG de cada par texto/fondo y gráfico/fondo en los dos temas (≥ 4.5:1 y ≥ 3:1).
- **Web (axe-core):** `apps/web/test/accessibility.test.tsx` corre las reglas WCAG 2.0–2.2 A y AA sobre la carga, "Ahora" e "Histórico". El contraste se excluye ahí porque jsdom no calcula estilos; lo cubre el punto anterior. Resultado del 5-nov: **0 violaciones**.
- **Móvil:** `apps/mobile/test/accessibility.test.tsx` revisa que todo control tocable de la carga, "Ahora", la hoja de ajustes, "Histórico" y la hoja de descarga tenga rol, nombre y área táctil ≥ 44 dp. Encontró "Abrir todas" (36 dp) y el chip "Mostrar sensores interiores" (32 dp); se corrigieron con `hitSlop` sin cambiar el diseño.
- **Árbol de accesibilidad del APK (6-nov):** `adb shell uiautomator dump --compressed` entrega los nodos que lee TalkBack (texto, `content-desc`, si es enfocable y su tamaño), sin tener que activarlo. Sin `--compressed` también salen los nodos ocultos al lector. En "Histórico" encontró:

- el campo "Últimos días" de 30 dp: ahora toda su píldora de 44 dp lo enfoca, y TalkBack solo lee el campo;
- las marcas de los ejes de las gráficas como nodos sueltos, ahora ocultas porque cada gráfica ya tiene su descripción;
- las cifras de Máxima/Mínima en tres nodos cada una, ahora uno ("Máxima: 28.0 °C, 3 nov, 11:15").

El test móvil cubre los tres casos. En "Ahora", `uiautomator` no termina ("could not get idle state"), porque la escena del cielo se anima sin parar; esa pantalla se revisa con el test y la lista manual. Los controles con `hitSlop` se ven más chicos en el volcado, que no incluye el `hitSlop`.

La lectura con TalkBack es manual y se hace en el APK, una vez por versión:

#	Paso	Se espera
1	Activar TalkBack y abrir la app	Anuncia la carga y luego "Condiciones actuales" con la temperatura y la condición
2	Deslizar por "Ahora"	Cada tarjeta se lee con su título y su valor y unidad; los íconos decorativos se saltan
3	Abrir "Ajustes" y cambiar idioma, unidades, tema y tamaño	Cada opción dice si está seleccionada; la hoja se cierra con "Listo" o con el gesto atrás
4	Ir a "Histórico" y cambiar el periodo y la resolución	Las resoluciones no permitidas se anuncian como deshabilitadas
5	Abrir y cerrar secciones	Cada sección dice si está abierta o cerrada; cada gráfica se lee con su resumen (rango y extremos)
6	Abrir "Descargar" y elegir un formato	Se lee cada formato con su descripción; se abre la hoja de compartir
7	Con la letra del sistema al máximo, repetir 2 y 5	El texto crece sin cortarse ni encimarse
8	Con "Quitar animaciones" activo, abrir "Ahora"	La escena del cielo queda fija

Operación

Despliegue (anexo operativo)

Este anexo es para el equipo que opera el sistema. Explica cómo se construye, publica y revierte cada app. No forma parte de la especificación funcional: la app no depende de él, y cambiar de hosting no cambia nada de los documentos 00–08.

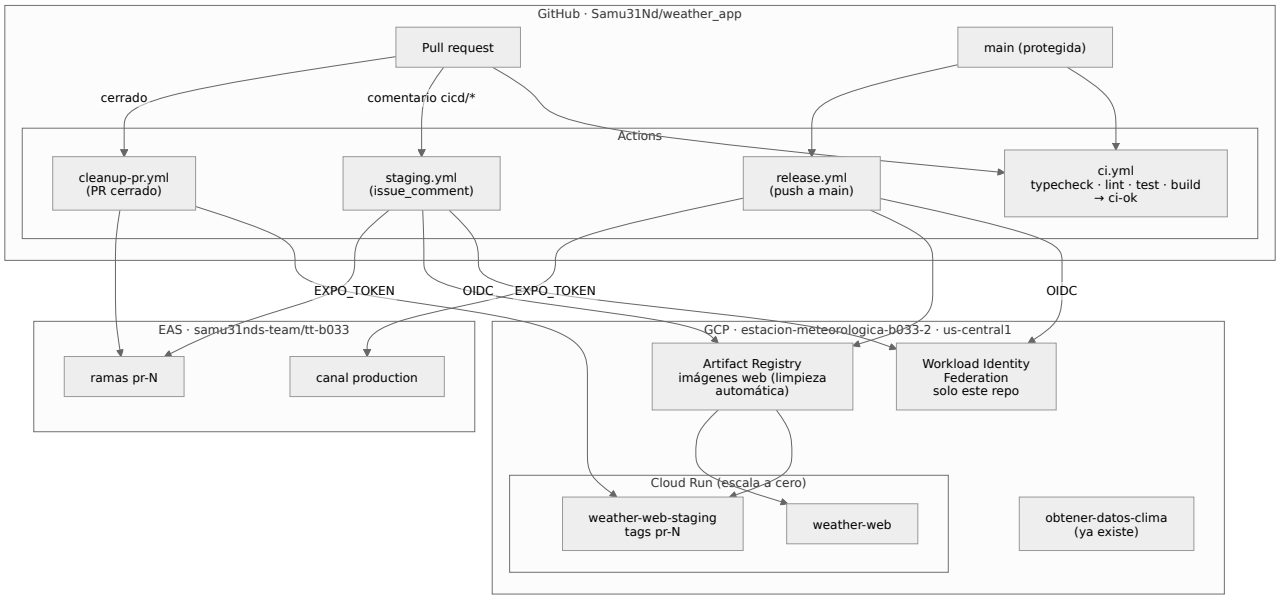
GitHub Actions orquesta todos los despliegues. Cloud Run (web) y EAS Update (móvil) solo son los lugares donde queda publicada cada app.

Entornos

Entorno	Web	Móvil	Cuándo se crea	Cuándo se borra
Local	`npm run web`	`npm run mobile` (Expo Go)	—	—
Staging de un PR	Revisión de `weather-web-staging` con el tag `pr-N`, sin tráfico. URL `https://pr-N---weather-web-staging-....run.app`	EAS Update en la rama `pr-N` (QR para Expo Go)	Al comentar `cicd/deploy-staging-web` o `cicd/deploy-staging-app` en el PR	Al cerrar o mergear el PR
Producción	`weather-web` con el 100 % del tráfico	EAS Update en el canal `production`	Al hacer merge a `main`, si cambió esa app	—

El **APK de producción** se genera a mano al final (ADR-0005).

Diagrama de despliegue



Reglas del flujo

- `main` exige el check `ci-ok` y **no permite saltarse la protección**, ni siquiera a los administradores. Se mergea con squash y la rama se borra al mergear.
- `ci-ok` resume todos los jobs. Así, los jobs que no corren porque su ruta no cambió no bloquean el merge.
- `staging.yml` vive en `main`, así que un PR no puede modificarlo para darse permisos. Solo actúa si:
  - quien comenta es OWNER, MEMBER o COLLABORATOR;
  - el comentario es exactamente `cicd/deploy-staging-web` o `cicd/deploy-staging-app`;
  - `ci-ok` del commit del PR está en verde.
- Producción **solo se publica desde `main`** y solo para la app que cambió (filtro de rutas: `apps/web/**`, `apps/mobile/**` y `packages/**`, que afecta a las dos).
- Los releases se marcan con `web-vAAAA.MM.DD[.n]` y `mobile-vAAAA.MM.DD[.n]`.
- **QR de producción móvil:** el release `mobile-v...` y el resumen del job de `release.yml` traen el QR de la actualización para abrirla en Expo Go, y también el enlace `exp://u.expo.dev/<proyecto>/group/<grupo>` para pegarlo en Expo Go (*Enter URL manually*) cuando no se puede escanear. El comentario de staging trae lo mismo. La app instalada (APK) no lo necesita: recibe sola las actualizaciones del canal `production`.
- Los workflows que se disparan por comentario corren con la versión de `main`. Si un PR agrega una variable al despliegue (como `EXPO_PUBLIC_HISTORY_API_URL`), su staging no la recibe hasta que se mergea.

Infraestructura

- La infraestructura de GCP (servicios Cloud Run, Artifact Registry, cuenta de servicio y WIF) se define como **Terraform en `proj-files/infra`**, junto al resto del sistema. No va en este repo.
- **Cuenta de servicio de despliegue:** permisos mínimos: `run.developer` sobre los dos servicios, `artifactregistry.writer` sobre el repositorio de imágenes e `iam.serviceAccountUser` sobre la cuenta de ejecución.
- **Los servicios web corren con una cuenta de servicio propia sin permisos de GCP.** Solo hacen

HTTP saliente.

Configuración

Nombre	Tipo	Uso
`EXPO_TOKEN`	secreto	Token del robot user de EAS (rol Developer)
`GCP_WIF_PROVIDER`	variable	Proveedor OIDC (`terraform output web_wif_provider`)
`GCP_DEPLOY_SA`	variable	Cuenta de servicio que despliega (`web_deployer_sa`)
`GCP_RUNTIME_SA`	variable	Cuenta de servicio con la que corre la web (`web_runtime_sa`)
`WEB_REGISTRY`	variable	Repositorio de imágenes (`web_registry`)
`NEXT_PUBLIC_API_URL`	variable	URL base de la consola. Se pasa al build y la usa el proxy `/api/live` del servidor
`NEXT_PUBLIC_API_CLOUD_URL`	variable	URL de la Cloud Function de histórico, para la web y (como `EXPO_PUBLIC_HISTORY_API_URL`) para el móvil
`MOBILE_WEATHER_STATION_URL`	variable	URL del dato en vivo para la app móvil: el proxy `https://weather-web-...run.app/api/live` (ADR-0007)

Los archivos son:

- `.github/workflows/{ci,staging,cleanup-pr,release}.yml`;
- `.github/actions/release-tag`;
- `apps/web/Dockerfile`, una imagen *standalone* de Next construida desde la raíz;
- `apps/mobile/eas.json`;
- `proj-files/infra/web_hosting.tf`.

Costos estimados (RNF-APP-11)

Recurso	Uso esperado	Capa gratuita	Costo esperado
Cloud Run (2 servicios, mínimo 0 instancias)	Tráfico de la comunidad ESCOM	2 M solicitudes y 180 k vCPU-s al mes	≈ 0 USD
Artifact Registry	$\leq$ 10 imágenes de unos 150 MB	0.5 GB	≈ 0–0.1 USD
GitHub Actions	Repo público	Ilimitado en repos públicos	0 USD
EAS Update	Pocos usuarios activos	Plan gratuito	0 USD
EAS Build	Solo el APK final, a mano	Cupo mensual gratuito	0 USD

APK de Android (local y a pedido)

El CI nunca compila binarios: solo publica EAS Updates, que no gastan el cupo de builds. El APK se compila en la máquina de quien lo necesite, sin límite:

```
mkdir -p ~/tmp-build
TMPDIR=~/tmp-build npm run build:android   # eas build --platform android --profile
↪  production --local
```

- Requiere JDK 17 y el Android SDK (por ejemplo, con Android Studio).
- **`TMPDIR` en disco.** `eas build --local` copia el repo y compila en `$TMPDIR` (unos 3.5 GB con Gradle). Si `/tmp` es un `tmpfs` pequeño (en la máquina de desarrollo es de 3.6 GB, en RAM), el build falla con `No space left on device`. Si un build falla así, borrar lo que dejó en `/tmp/<usuario>/eas-build-local-nodejs`.
- Genera el `.apk` en `apps/mobile/`.
- Solo hace falta un APK nuevo si cambian las dependencias nativas o la versión de la app (`runtimeVersion: appVersion`). Los demás cambios llegan por EAS Update al canal `production`.
- El widget de Android (ADR-0011) es código nativo: solo aparece en el APK, y agregar o cambiar widgets en `app.json` requiere generar un APK nuevo.
- **Las URLs del APK salen de las variables de entorno de EAS, no de `.env`.** `eas build --local` compila una copia del repo que respeta `.gitignore`, así que `.env` no llega a la build. Los perfiles `preview` y `production` de `eas.json` declaran su `environment`, y EAS inyecta `EXPO_PUBLIC_WEATHER_STATION_URL` (el proxy HTTPS; el APK no permite HTTP) y `EXPO_PUBLIC_HISTORY_API_URL` del proyecto `@samu31nds-team/tt-b033`. Son URLs públicas (visibilidad *plaintext*), no secretos. Se revisan con `eas env:list --environment production`, se cambian con `eas env:update` y se comprueban con `eas env:exec production 'echo $EXPO_PUBLIC_HISTORY_API_URL'`.
- **Identificador del APK:** `android.package` es `mx.ipn.escom.ttb033` (definitivo desde el 5-nov). Los APK de prueba anteriores usaban `com.samu31nd.ttb033`: Android los trata como otra app, así que hay que desinstalarlos.
- Para comprobar un APK sin instalarlo: `unzip -p <apk> assets/index.android.bundle | grep -ac run.app` debe dar más de 0.

Rollback

- **Web:** `gcloud run services update-traffic weather-web --to-revisions=<anterior>=100`. Tarda menos de 1 min.
- **Móvil:** `eas update:republish` de la actualización anterior en el canal `production`.
- En los dos casos, después se revierte el commit en `main` con un PR normal.

Mantenimiento y traspaso

Qué hace falta para que el sistema siga funcionando cuando cambie quien lo mantiene: qué cuentas y accesos hay que entregar, qué tareas se repiten, cómo actualizar las piezas que envejecen y qué hacer cuando algo falla. Complementa el anexo operativo 09 · Despliegue, que describe el CI/CD en detalle.

Inventario: de qué depende la app

Pieza	Dónde vive	Quién la administra hoy	Qué pasa si se pierde el acceso
Código	GitHub `Samu31Nd/weather_app`	El autor (dueño del repo)	Nadie puede mergear a `main` ni cambiar variables del CI; se puede hacer *fork*, pero hay que reconfigurar todo el CI/CD
Web	Cloud Run, proyecto GCP `estacion-meteorologica-b033-2` (`us-central1`), servicios `weather-web` y `weather-web-staging`	Cuenta de facturación del proyecto	La web sigue en línea mientras la facturación siga activa, pero no se puede desplegar
Infraestructura como código	Terraform en `proj-files/infra` (fuera de este repo)	El autor	Hay que recrear a mano Cloud Run, Artifact Registry y la federación de identidad
Móvil (actualizaciones y builds)	EAS, proyecto `@samu31nds-team/tt-b033`	Equipo de Expo `samu31nds-team`	Las apps instaladas siguen funcionando con la última versión, pero no reciben actualizaciones
Token del CI para EAS	Secreto `EXPO_TOKEN` del repo (robot user con rol Developer)	Equipo de Expo	Fallan los jobs móviles de `staging.yml` y `release.yml`
Dato en vivo	Consola Davis Vantage Vue en la red de ESCOM	Responsable de la estación	La app muestra "sin conexión"; el histórico sigue
Histórico	Cloud Function `obtener-datos-clima` y el pipeline ELT (en `proj-files`)	Responsable del pipeline	"Histórico" deja de cargar días nuevos; la caché local conserva los ya vistos
Diseño	Canvas de Claude Design "Estación meteorológica" (ver `AGENTS.md`)	El autor	Sin él no se pueden extraer escenas nuevas; el arte actual ya está en `@tt/sky-art`

Lista para el traspaso

Al entregar el proyecto a una nueva persona responsable (o a la ESCOM):

- ☐ **GitHub:** transferir el repositorio a una organización institucional o agregar a la nueva persona como

admin. Al transferirlo, revisar que la federación de identidad de GCP (WIF) siga apuntando al nombre nuevo del repo: está restringida a este repo (`proj-files/infra/web_hosting.tf`).

- ☐ **GCP:** dar a la nueva persona el rol *Owner* (o *Editor* + *Billing*) del proyecto, y confirmar quién paga la cuenta de facturación. Crear la alerta de presupuesto de 1 USD, que sigue pendiente.
- ☐ **Expo/EAS:** invitarla al equipo `samu31nds-team` como *Owner*, o transferir el proyecto a otra cuenta. Si cambia de cuenta, generar un `EXPO_TOKEN` nuevo y guardarlo en los secretos del repo.
- ☐ **Terraform:** entregar `proj-files/infra` y su estado (`*.tfstate`) por un canal privado. **Nunca** por el repo: contiene datos sensibles.
- ☐ **Consola Davis:** contacto de quien tiene acceso físico y de red a la estación.
- ☐ **Variables:** revisar que las variables de GitHub (*Settings* → *Secrets and variables* → *Actions*) y las de EAS (`eas env:list --environment production`) sigan siendo correctas.
- ☐ **Firma del APK:** EAS guarda el *keystore* de Android del proyecto. Sin él, un APK nuevo no puede actualizar al instalado (Android lo trata como otra app). Descargar un respaldo con `eas credentials`.

Tareas periódicas

Cada...	Tarea	Cómo
Mes	Revisar que producción responde	Abrir la web; `curl -s <web>/api/live` debe dar JSON; abrir "Histórico" de 7 días
Mes	Revisar costos	Consola de facturación de GCP (esperado ≈ 0 USD, ver 09)
Semestre	Revisar que las fuentes no cambiaron de forma	`npm run test:contract --workspace=@tt/core` con las URLs reales
Semestre	Actualizar dependencias menores	Abajo
Año (o cuando Expo Go deje de abrir la app)	Migrar al SDK nuevo de Expo	Abajo
Versión de la app	Revisión con TalkBack	Lista manual de 08

Actualizar dependencias

```
npm outdated                          # en la raíz: qué está atrasado en cada workspace
npm update                            # solo versiones compatibles (^ y ~)
npm run typecheck && npm test && npm run build --workspace=web
```

- Para la app móvil, usa `npx expo install --check` desde `apps/mobile`: Expo sabe qué versiones van con el SDK y las versiones "más nuevas" de npm suelen romperlo.
- React, `@types/react` y TypeScript deben tener **la misma versión en las dos apps**; si no, npm instala dos copias y aparecen errores de tipos raros.
- Si cambia una dependencia nativa (cualquier cosa con código Android/iOS), la app instalada no la recibe por EAS Update: hace falta un APK nuevo y subir `version` en `app.json` (`runtimeVersion` sigue a `appVersion`).

Actualizar el SDK de Expo

Expo Go solo abre proyectos del SDK más reciente, así que esto se vuelve obligatorio aproximadamente una vez al año. La migración de 57 a 58 dejó estas lecciones:

1. Lee la guía de migración y el *changelog* del SDK nuevo en https://docs.expo.dev.

2. En `apps/mobile`: `npx expo install expo@^<nuevo>` y luego `npx expo install --fix`.
3. **Regenera `package-lock.json` desde cero** (borra `node_modules` y el lock en la raíz, `npm install`). Si no, npm deja copias viejas de reanimated/worklets en la raíz y TypeScript toma la versión vieja de React Native.
4. Alinea `react`, `react-dom` y `@types/react` en `apps/web` con la versión que pide el SDK.
5. `npx expo-doctor` desde `apps/mobile` y corrige lo que reporte.
6. Corre todo (`typecheck`, `test`, `build` de la web, `npx expo export --platform android`) y abre la app en Expo Go y en un APK de prueba. Revisa APIs que suelen cambiar (en el 58 cambió `tabBar` de Expo Router).
7. Sube `version` en `app.json`: el SDK nuevo cambia el código nativo, así que hace falta un APK nuevo.
8. Actualiza las versiones en `AGENTS.md`, `apps/mobile/AGENTS.md` y 12 · Implementación.

Cambios de configuración frecuentes

Cambio	Dónde
Cambió la IP o URL de la consola	Variable `NEXT_PUBLIC_API_URL` del repo (la usa el build de la web) y volver a desplegar la web (merge a `main` o *re-run* de `release.yml`). La móvil no cambia: usa el proxy
Cambió la URL de la Cloud Function	Variable `NEXT_PUBLIC_API_CLOUD_URL` del repo **y** `EXPO_PUBLIC_HISTORY_API_URL` en EAS (`eas env:update`); redesplegar web y publicar una actualización móvil
Cambió la URL de la web	`MOBILE_WEATHER_STATION_URL` del repo y `EXPO_PUBLIC_WEATHER_STATION_URL` en EAS
El pipeline corrigió datos viejos (por ejemplo, D5)	Subir `HISTORY_CACHE_PREFIX` en `cached-history.repository.ts` a `wx-history-v2:`
Venció o se filtró `EXPO_TOKEN`	Crear otro token del robot user en expo.dev y reemplazar el secreto del repo
Renovar la vista previa de staging	Comentar otra vez `cicd/deploy-staging-web` o `cicd/deploy-staging-app` en el PR

Cuando algo falla en producción

Síntoma	Dónde mirar	Qué hacer
"Ahora" dice "sin conexión" para todos	`curl <web>/api/live`: 504 = la consola no responde; 502 = respondió con error; 500 = falta la URL	Revisar la consola (energía, red del IPN) o la variable de la URL
"Ahora" funciona en local pero no en producción	Logs del servicio en Cloud Run (`gcloud run services logs read weather-web`)	Ver si la URL llegó al build
"Histórico" no carga	Abrir la URL de la Cloud Function con `?vista=vw_clean_weather&date_start=...&date_end=...`	Si falla ahí, es del pipeline, no de la app
El histórico tiene huecos	06 · Datos (D7, D8)	Días perdidos o el día de hoy: es lo esperado; se dibujan como cortes
La rosa de los vientos se ve rara	D5	Pendiente del pipeline (abajo)
El CI falla sin cambios propios	La pestaña *Actions* del PR	Volver a correr; si persiste, puede ser una acción o versión de Node que cambió

Síntoma	Dónde mirar	Qué hacer
Un despliegue rompió producción	—	*Rollback* según 09 y luego revertir el commit con un PR
La app instalada no recibe un cambio	Canal `production` en expo.dev	Si el cambio es nativo, necesita APK nuevo; si no, revisar que `release.yml` publicó

Pendientes conocidos

Lo que quedó fuera del alcance y conviene retomar:

- **D5 en el pipeline:** en las filas de la captura automática, `wind_dir` y `hi_dir` vienen intercambiados. Se corrige en el `INSERT` del pipeline y migrando esas filas; después, subir el prefijo de la caché.
- **D12 en la consola:** su zona horaria está en UTC−5. La app ya lo compensa; si alguien la corrige a UTC−6, la app sigue funcionando sin cambios.
- **Alerta de presupuesto** en GCP (1 USD): la debe crear quien administre la facturación.
- **Pruebas e2e** con Playwright: están en la estrategia de 08 pero aún no se escriben.
- **Ideas:** fases de la luna calculadas por fecha (la consola no las publica); un paquete `@tt/react` con los hooks compartidos; iOS (el código es multiplataforma, pero nunca se compiló ni probó un binario de iOS).

Glosario

Términos que aparecen en la guía y en el código. Los de meteorología están pensados para quien viene de computación, y los de software para quien viene de otra área.

Meteorología y la estación

Término	Significado
Davis Vantage Vue	Modelo de la estación de ESCOM ("Liz"). Tiene sensores exteriores (temperatura, humedad, viento, lluvia) y una **consola** interior con presión, temperatura y humedad interiores
`wflexp.json`	Archivo JSON que la consola publica en la red local con la lectura actual
Beaufort	Escala de 0 (calma) a 12 (huracán) que clasifica el viento por sus efectos
Ráfaga	Velocidad máxima del viento en un intervalo corto; suele ser mayor que el promedio
Rumbo / punto cardinal	Dirección *desde* donde sopla el viento, en grados (0° = norte) o en 16 puntos (N, NNE, NE…)
Rosa de los vientos	Gráfica circular que muestra con qué frecuencia sopló el viento desde cada dirección
Punto de rocío	Temperatura a la que el aire se saturaría de humedad. Cuanto más cerca de la temperatura, más húmedo se siente; si casi coinciden, puede haber niebla
Sensación térmica	Temperatura que se percibe por el efecto del viento (enfriamiento) o de la humedad (índice de calor)
Presión a nivel del mar	Presión corregida como si la estación estuviera al nivel del mar, para compararla con otras estaciones. En ESCOM (2240 m) la presión real es mucho menor (D3)
Tendencia de presión	Si la presión sube o baja en las últimas 3 h; una bajada rápida suele anunciar mal tiempo
Intensidad de lluvia	Lluvia por hora (mm/h) en este momento, distinta de la lluvia acumulada del día
Pronóstico Davis	Regla (1 de 195) que la consola elige a partir de presión, viento y otras señales; la app la traduce a texto e ícono
CDMX (UTC−6)	Zona horaria en la que la app calcula días, semanas y horas. México abolió el horario de verano en 2022
D1…D13	Identificadores de las rarezas conocidas de los datos, en 06 · Datos

Software y arquitectura

Término	Significado
Monorepo	Un solo repositorio con varios proyectos (aquí, dos apps y cuatro paquetes) que comparten código
Workspace (npm)	Cada proyecto dentro del monorepo; npm los instala juntos y los enlaza entre sí
Arquitectura hexagonal (puertos y adaptadores)	Forma de organizar el código en la que la lógica del negocio no sabe nada de pantallas, red ni almacenamiento; esas cosas se conectan desde afuera
Dominio	Las reglas del problema (convertir unidades, decidir la escena del cielo), escritas sin depender de la tecnología
Puerto	Interfaz que el dominio define para pedir algo al exterior ("dame la lectura actual")
Adaptador	Implementación concreta de un puerto ("pídela por HTTP a esta URL")
Caso de uso	Una acción completa que la app ofrece (`GetHistory`), que coordina dominio y puertos
Mapper	Función que traduce el formato de una fuente externa al modelo de la app; aquí se corrigen las rarezas
DTO	*Data Transfer Object*: el tipo que describe los datos tal como llegan, antes del mapper
Modelo de vista	Objeto con todo lo que una pantalla necesita ya calculado (`CurrentConditions`, `HistoryView`)
Composition root / contenedor	El único lugar donde se crean y conectan todas las piezas (`createCore`)
Inyección de dependencias (DI)	Pasarle a un objeto lo que necesita en lugar de que lo cree él; permite usar dobles en las pruebas
Doble de prueba	Implementación falsa (repositorio en memoria, reloj fijo) que sustituye a la real en una prueba
Fixture	Archivo con datos reales guardado para que las pruebas los usen
Prueba de contrato	Prueba que consulta las fuentes reales para comprobar que siguen entregando la forma esperada
Componente (React)	Función que recibe datos (*props*) y devuelve lo que se dibuja
Hook (React)	Función `use...` que da estado o efectos a un componente (`useLiveWeather`)
Provider / contexto (React)	Mecanismo para que muchos componentes lean un mismo valor (el núcleo, las preferencias) sin pasarlo a mano
Route handler	Función de Next.js que responde a una URL del servidor (`/api/live`)
Proxy	Servidor intermedio que hace la petición en nombre del cliente
CORS	Regla del navegador que impide a una página leer respuestas de otro dominio que no lo autorice
Token de diseño	Valor con nombre del sistema visual (un color, un tamaño) que comparten diseño y código
WCAG / AA	Pautas de accesibilidad web; el nivel AA pide, entre otras cosas, contraste de 4.5:1 en texto
TalkBack	Lector de pantalla de Android
Metro	Empaquetador de JavaScript de React Native (el equivalente a webpack/Vite)

Término	Significado
Hermes	Motor de JavaScript que corre la app en Android
Expo Go	App de Expo que abre proyectos en desarrollo sin compilarlos
APK	Archivo instalable de una app de Android
CNG	*Continuous Native Generation*: el proyecto nativo (`/android`) se genera en cada build a partir de `app.json`
EAS	*Expo Application Services*: la nube de Expo para compilar (Build) y publicar actualizaciones (Update)
OTA / EAS Update	*Over the air*: actualizar el JavaScript de la app instalada sin reinstalarla
Worklet	Función que reanimated ejecuta en el hilo de UI, para animaciones fluidas
CI/CD	Integración continua (correr pruebas en cada cambio) y despliegue continuo (publicar automáticamente)
Cloud Run	Servicio de Google Cloud que corre un contenedor y escala a cero cuando nadie lo usa
WIF	*Workload Identity Federation*: GitHub Actions se autentica en GCP sin guardar llaves
Terraform	Herramienta para describir la infraestructura como código
ADR	*Architecture Decision Record*: documento corto que registra una decisión, su contexto y sus consecuencias
Conventional Commits	Convención para escribir mensajes de commit con tipo y alcance (`fix(mobile): ...`)
Squash merge	Unir un PR a `main` como un solo commit
Parquet	Formato de archivo por columnas para análisis de datos (pandas, DuckDB, Spark)
RF / RNF / CU	Requerimiento funcional, requerimiento no funcional, caso de uso (con sus IDs `RF-APP-NN`, `RNF-APP-NN`, `CU-NN`)

Registro de decisiones (ADR)

Cada decisión de arquitectura tiene un archivo con su contexto, la decisión y sus consecuencias. Una decisión no se borra: si cambia, se escribe un ADR nuevo que la **reemplaza** y se marca el viejo.

ADR	Decisión	Estado
0001	Un monorepo con núcleo compartido	Aceptada
0002	Next.js para web y Expo para móvil, en vez de solo Expo	Aceptada
0003	npm workspaces en vez de pnpm	Aceptada
0004	Web en Cloud Run, desplegada desde GitHub Actions	Aceptada
0005	Móvil: staging y producción por OTA; APK a mano al final	Aceptada
0006	Gráficas en SVG propio con geometría compartida	Aceptada
0007	Proxy para el dato en vivo, también para el móvil	Aceptada
0008	Especificación en `docs/` antes del código y antes del LaTeX	Aceptada
0009	El arte del diseño se extrae del canvas con un script, no se copia a mano	Aceptada
0010	Las animaciones del diseño se describen como datos y móvil las reproduce con reanimated	Aceptada
0011	Widget de Android con react-native-android-widget, solo en el APK	Aceptada
0012	Exportar a Parquet con hyparquet-writer dentro del núcleo	Aceptada
0013	Caché persistente del histórico por día	Aceptada

ADR-0001 · Un monorepo con núcleo compartido

- **Estado:** Aceptada (2026-10-20)

Contexto

Había dos apps en dos repos: `weather_app` (Expo) y `weather-dashboard` (Next.js). Cada una tenía su propio `WeatherEntity`, sus mappers y sus conversores. Ya habían divergido: un error de máximos y mínimos estaba corregido en una y no en la otra. Mantener dos copias de la lógica obliga a sincronizarlas a mano.

Decisión

Un solo repo (`Samu31Nd/weather_app`) con `apps/web`, `apps/mobile` y `packages/*`. La lógica de dominio, los casos de uso y los adaptadores HTTP viven una sola vez en `packages/core`. Las dos historias de git se conservan: `git mv` para la app móvil y `git subtree add` para la web.

Consecuencias

- Una corrección en `core` llega a las dos apps a la vez y se prueba una sola vez.

- El CI tiene que filtrar por rutas para no compilar y desplegar todo en cada cambio.
- El repo `weather-dashboard` se archiva.

ADR-0002 · Next.js para web y Expo para móvil, en vez de solo Expo

- **Estado:** Aceptada (2026-10-20)

Contexto

Expo puede generar la web con react-native-web. Al probarlo, el cambio manual de tema claro/oscuro no funcionaba en web: uniwind y `Appearance` no se reflejaban. Además, el dato en vivo necesita un servidor por CORS, y la app web de Next ya tenía el histórico, los route handlers de proxy y Tailwind web nativo.

Decisión

- **Web:** Next.js (App Router), con CSS real. El tema se aplica con una clase en `<html>` y variables CSS.
- **Móvil:** Expo.
- Las dos comparten `packages/core`, `packages/tokens` y `packages/i18n`, pero **cada una tiene sus propios componentes de UI**.

Consecuencias

- Se escriben dos UIs, pero cada una aprovecha su plataforma: en web, CSS, media queries y `prefers-*`; en móvil, reanimated y gestos.
- El tema web deja de depender de react-native-web.
- La consistencia visual se cuida con tokens compartidos y con los artboards del diseño (RNF-APP-04).

ADR-0003 · npm workspaces en vez de pnpm

- **Estado:** Aceptada (2026-10-20). Reemplaza la propuesta inicial de usar pnpm.

Contexto

El plan proponía pnpm, pero no está instalado en la máquina de desarrollo. Expo y Next funcionan con npm workspaces sin configuración extra, y Metro ya detecta el monorepo.

Decisión

Usar npm workspaces, con un solo `package-lock.json` en la raíz. React, `@types/react` y TypeScript quedan en la misma versión en las dos apps para que no se dupliquen.

Consecuencias

- No hay que instalar nada global y el CI usa la caché nativa de `actions/setup-node`.
- npm hace *hoisting*: una dependencia no declarada en un workspace puede funcionar por accidente. El typecheck en CI y las revisiones de PR lo vigilan.

ADR-0004 · Web en Cloud Run, desplegada desde GitHub Actions

- **Estado:** Aceptada (2026-10-21)

Contexto

La web necesita un servidor por el proxy del dato en vivo. Se querían staging por PR bajo demanda (con un comentario), producción separada, checks obligatorios y costo cercano a cero. Las alternativas eran Vercel (gratis y simple, pero otra cuenta y otro proveedor) y Cloud Run en el proyecto GCP que ya existe.

Decisión

- Cloud Run en `estacion-meteorologica-b033-2`, con dos servicios: `weather-web` y `weather-web-staging`. Escalan a cero.
- Cada PR tiene una revisión etiquetada `pr-N`, sin tráfico.
- Actions se autentica con **Workload Identity Federation**, sin llaves JSON.
- La infraestructura se define como Terraform en `proj-files/infra`.

Consecuencias

- Todo el sistema queda en un solo proveedor y un solo proyecto, lo que es coherente con la tesis.
- Sin tráfico el costo es prácticamente 0. Artifact Registry necesita una política de limpieza.
- Hay más configuración inicial que con Vercel (WIF y cuentas de servicio).

ADR-0005 · Móvil: staging y producción por OTA; APK a mano al final

- **Estado:** Aceptada (2026-10-21)

Contexto

EAS Build tiene un cupo mensual limitado en el plan gratuito. Mientras la app está en desarrollo, casi todos los cambios son de JavaScript.

Decisión

- **Staging:** `eas update --branch pr-N`, que se abre en Expo Go con un QR. No compila nada.
- **Producción:** `eas update --channel production`.
- El APK de producción se genera **a mano, al final**, cuando la app esté lista. No hay builds en el CI.
- El CI se autentica con el token de un *robot user* de la organización `samu31nds-team`.

Consecuencias

- No se gasta el cupo de builds.
- Mientras no exista el APK, para probar la app hay que usar Expo Go. Por eso solo se usan módulos que Expo Go soporta, o se documenta la excepción.

ADR-0006 · Gráficas en SVG propio con geometría compartida

- **Estado:** Aceptada (2026-10-21)

Contexto

La web usaba MUI X Charts (con emotion y MUI completos) y el móvil SVG a mano. El diseño pide gráficas específicas (banda mín./máx., rosa de los vientos, acumulado de lluvia, eje de presión ajustado) en las dos plataformas. No hay una librería que funcione igual en web y en React Native.

Decisión

- Las escalas, los ticks, los paths y la rosa de los vientos se calculan en `packages/core/charts` como funciones puras.
- Cada app solo dibuja: `<svg>` en web y `react-native-svg` en móvil.
- Se quitan MUI y recharts.

Consecuencias

- Las gráficas son idénticas en las dos plataformas y su geometría tiene tests unitarios.
- El bundle web es más chico.
- La interacción (tooltips, cursor) hay que escribirla para cada plataforma.

ADR-0007 · Proxy para el dato en vivo, también para el móvil

- **Estado:** Aceptada (2026-10-21)

Contexto

La consola Davis solo sirve HTTP, sin cabeceras CORS, desde una IP del IPN. Un navegador servido por HTTPS no puede llamarla. En móvil sí se podría llamar directo, pero para eso hay que permitir tráfico *cleartext* en Android e iOS y exponer la IP en el bundle.

Decisión

- Un route handler `/api/live` en la app web consulta la consola desde el servidor:
  - host fijo por variable de entorno;
  - timeout de 8 s;
  - caché de unos 10 s;
  - no reenvía las cabeceras del cliente.
- El móvil usa ese mismo proxy por defecto (`EXPO_PUBLIC_WEATHER_STATION_URL`). La conexión directa queda como opción de desarrollo en `.env`: Expo Go permite HTTP, y el APK no lleva excepción de *cleartext*.

Consecuencias

- Hay HTTPS de punta a punta para el cliente, la IP de la consola no queda expuesta y no hay excepciones de *cleartext* en producción.
- Si Cloud Run se cae, el móvil también pierde el dato en vivo. Se acepta: hay un estado "sin conexión" y el SLA de Cloud Run es alto.
- La caché evita que muchos usuarios a la vez saturen la consola.

ADR-0008 · Especificación en `docs/` antes del código y antes del LaTeX

- **Estado:** Aceptada (2026-10-21)

Contexto

Los requerimientos de la tesis (cap. 6) solo contemplan la web y están desactualizados. Se iba a reescribir mucho código sin una especificación que diera el porqué de cada parte.

Decisión

- Antes de las fases de código se escribe la especificación en `docs/`, en Markdown y con diagramas Mermaid: visión, requerimientos, casos de uso, arquitectura, dominio, flujos, datos, interfaz, pruebas, despliegue y ADR.
- El código, los PR y los tests citan sus IDs.
- Al final, la tesis toma una versión resumida (cuerpo y anexos), porque su tema principal es el procesamiento de datos.

Consecuencias

- Cada componente y cada test se rastrean hasta un requerimiento (matriz en `08-calidad-pruebas.md`).
- La especificación se actualiza en el mismo PR que la cambia.

ADR-0009 · El arte del diseño se extrae del canvas con un script

- **Estado:** Aceptada (2026-10-24)

Contexto

Las 21 escenas del cielo y la ilustración de carga son SVG con animaciones CSS dentro de los artboards del canvas de Claude Design. Copiarlos a mano a cada app es lento, propenso a errores y se desactualiza cuando cambia el diseño. Además, sus clases (`.drift`, `.rain`, `.spin`) e ids chocarían con los estilos de la app.

Decisión

- `packages/sky-art/scripts/extract-from-design.mjs` lee los `project/Sky*.dc.html` descargados del canvas y genera `src/scenes.generated.ts` y `src/styles.generated.ts`.
- Las clases y keyframes llevan el prefijo `sky-`, y los ids el nombre de la escena.
- Las etiquetas visibles no se copian: las pone `@tt/i18n` en el idioma elegido.
- Un test verifica que estén las 21 escenas, que las referencias `url(#...)` sean válidas, que los ids no se repitan y que toda animación se apague con `prefers-reduced-motion`.
- La ilustración de carga se extrajo de la misma forma, con el prefijo `wx-load-`.

Consecuencias

- Si el diseño cambia, se vuelve a descargar del canvas y se corre `npm run extract --workspace=@tt/sky-art`.
- El SVG se inyecta como contenido propio (`dangerouslySetInnerHTML`). El test garantiza que no trae scripts ni manejadores de eventos.

- En móvil el mismo SVG se dibuja con `react-native-svg` y las animaciones se reproducen como datos (ADR-0010).

ADR-0010 · Las animaciones del diseño se describen como datos para móvil

- **Estado:** Aceptada (2026-10-24)

Contexto

El arte del diseño (21 escenas del cielo y la ilustración de carga, ADR-0009) se anima con CSS: nubes a la deriva, lluvia, rayos que giran, estrellas que titilan, el anemómetro... En web basta con la hoja de estilos. React Native no tiene CSS: `react-native-svg` dibuja el SVG, pero las animaciones hay que hacerlas con reanimated. Reescribirlas a mano en cada componente duplicaría el diseño y se desfasaría de la web.

Decisión

- `@tt/sky-art` incluye un lector pequeño del SVG (`parseSvgFragment`) y la transcripción de cada `@keyframes` como datos (`resolveMotion`): pistas por propiedad, duración, curva, dirección, retraso y punto de giro, con las mismas reglas de selección que el CSS (`.sky-rain path:nth-child(3n)`, `.sky-windy .sky-drift`, `--rs`, `--dx`, `--spin`).
- `sampleMotion` y `motionMatrix` son funciones puras marcadas `'worklet'`: reanimated las ejecuta en el hilo de UI y vitest las prueba como cualquier función.
- En móvil, `SvgArt` recorre el árbol, pone los colores de la paleta y anima cada elemento con un único reloj por dibujo (`useFrameCallback`). Con "reducir movimiento" activado en el sistema, el arte queda quieto.
- Un test compara duraciones, curvas y direcciones con las hojas CSS extraídas, y verifica que toda clase animada del arte tenga su transcripción.

Consecuencias

- Web y móvil muestran las mismas animaciones con una sola fuente: el canvas de diseño.
- Diferencias aceptadas: un retraso positivo se toma como desfase de fase (en CSS el elemento espera quieto la primera vez) y `transform-box: fill-box` usa el centro aproximado de la figura.
- Si el diseño agrega una animación nueva, el test de cobertura falla hasta transcribirla en `motion.ts`.

ADR-0011 · Widget de Android con react-native-android-widget

- **Estado:** Aceptada (2026-10-25)

Contexto

Se quiere un widget en la pantalla de inicio con el clima de la estación, solo en el APK (Android). Las opciones eran escribirlo en Kotlin con Glance o RemoteViews, o usar una librería que lo describa en JSX desde el mismo código de la app. iOS queda fuera: necesita WidgetKit en Swift y una cuenta de desarrollador de pago.

Decisión

- `react-native-android-widget` (config plugin de Expo): el widget se escribe con `FlexWidget`, `TextWidget` y `SvgWidget`, y un manejador en JS headless lo dibuja al agregarlo, cada 30 min (el mínimo de Android), al redimensionarlo y al tocar ↻. La app también lo redibuja cada vez que recibe un dato nuevo.

- Usa el mismo núcleo que la app (`createMobileCore`, preferencias de idioma y unidades). Lo propio del widget vive en los paquetes: reglas de ícono y dato contextual en `@tt/core` (`widgetKind`, `widgetDetail`), textos en `@tt/i18n` (`widgetText`), colores en `@tt/tokens` (`WIDGET_PALETTES`, con contraste AA) e íconos en `@tt/sky-art` (`widgetIconSvg`).
- Se entregan las versiones clara y oscura; Android elige según el tema del sistema. No se fija la fuente: el texto usa la del usuario y respeta su tamaño de letra.
- La librería falla al importarse sin su módulo nativo, así que solo se carga en Android fuera de Expo Go (`WIDGETS_AVAILABLE`), con `require` diferido.

Consecuencias

- El widget solo se ve en el APK (`npm run build:android`); Expo Go y el staging por OTA no lo muestran.
- Agregar o cambiar widgets cambia el código nativo: requiere un APK nuevo, no basta una actualización OTA.
- Si la consola no responde, el widget dibuja completa la última lectura guardada (por él o por la app), con la temperatura en gris y su hora; "sin conexión" queda solo si nunca hubo una (ajuste del 5-nov).
- La librería dibuja el widget desde un trabajo de WorkManager 2.8.1. En la build de release, R8 (modo completo) quitaba el constructor de `OverwritingInputMerger`, el trabajo fallaba y el widget quedaba transparente. La regla `-keepclassmembers class * extends androidx.work.InputMerger { public <init>(); }` se agrega con `expo-build-properties` en `app.json`.
- La librería llama a los componentes del widget como funciones simples. El React Compiler (activo en la app) les agrega hooks de memorización y el render fallaba con "Invalid Hook Call"; `weather-widget.tsx` lleva `'use no memo'` para excluirlo.

ADR-0012 · Exportar a Parquet con hyparquet-writer dentro del núcleo

- **Estado:** Aceptada (2026-11-05)

Contexto

Se pidió descargar el histórico también en Parquet, el formato columnar que leen directo pandas, polars, DuckDB y Spark, con tipos y compresión. Escribir Parquet a mano (Thrift, páginas, Snappy) no tiene sentido; hace falta una librería. Las dos apps exportan con `exportHistory` de `@tt/core`, y la regla de dependencias (ADR-0001, RNF-APP-08) no deja importar librerías en el núcleo.

Decisión

- `hyparquet-writer` (versión exacta en `packages/core/package.json`) dentro de `exportHistory`. Es JS puro, sin dependencias nativas ni WebAssembly, y solo usa `ArrayBuffer`/`TextEncoder`: corre igual en el navegador, en Node (tests) y en Hermes. `parquet-wasm` y `duckdb-wasm` se descartaron porque Hermes no ejecuta WebAssembly.
- El test de arquitectura (`packages/core/test/architecture.test.ts`) la permite en una lista explícita; cualquier otra librería sigue fallando. La regla de fondo no cambia: el núcleo no usa React ni APIs de plataforma.
- El archivo tiene las mismas columnas que el CSV, más `time_utc` como `TIMESTAMP`; las cifras van como `DOUBLE`, los huecos como `null`, y la resolución y las unidades en los metadatos clave-valor. Compresión Snappy.

Consecuencias

- `HistoryExport.content` pasa a ser `string | Uint8Array`. La web arma el `Blob` sin `charset` para binarios y el móvil escribe los bytes con `expo-file-system`.
- La librería publica una entrada para Node que importa `fs`. Metro no usa la condición `browser` en Android, así que `apps/mobile/metro.config.js` la agrega solo para `hyparquet*`; jest la transforma (`transform IgnorePatterns`).
- Los tests leen el archivo de vuelta con `hyparquet` (misma versión que usa el escritor).

ADR-0013 · Caché persistente del histórico por día

- **Estado:** Aceptada (2026-11-05)

Contexto

La pantalla "Histórico" guardaba en memoria cada periodo ya pedido, pero la caché se perdía al recargar la web o al salir de la pestaña en móvil, y era por periodo exacto: 24 h, 7 días y 30 días comparten días y los volvían a pedir. Un periodo de 90 días son 9 consultas a la Cloud Function. Casi todo el histórico ya no cambia: solo los últimos días pueden completarse, porque los datos llegan con un día de retraso (D7).

Decisión

- `CachedHistoryRepository` en `@tt/core` decora el `HistoryRepository` de HTTP y lo arma `createCore`, así web y móvil lo usan igual. Guarda **por día de CDMX** y solo pide los días que faltan, agrupados en tramos contiguos de 10 días o menos.
- Vigencia: un día que se pidió más de 48 h después de cerrar es definitivo y no vence; uno más reciente vence a la hora. También se guardan los días sin lecturas (huecos conocidos). Si la fuente falla no se guarda nada.
- Persiste en el mismo almacén clave-valor de las preferencias (`localStorage` en web, AsyncStorage en móvil), en claves `wx-history-v1:YYYY-MM-DD` con una fila de valores por muestra (≈ 7 KB por día). Tope de 200 días; al pasarse se borran los usados hace más tiempo.
- Un almacén que falla o una entrada dañada cuentan como "no guardado": se vuelve a pedir. Un rango que no empieza y termina a medianoche de CDMX pasa directo a la fuente.

Consecuencias

- Cambiar de periodo, volver a la pantalla o reabrir la app casi no consulta la Cloud Function; los 200 días del histórico completo caben en ≈ 1.4 MB, debajo del límite de `localStorage` (~5 MB).
- Si se corrige un dato viejo en el pipeline (p. ej. D5), los clientes no lo ven hasta borrar la caché: cambiar el prefijo de versión (`wx-history-v2:`) la invalida.
- La caché devuelve el rango medio abierto `[inicio, fin)`: ya no llega la muestra de la medianoche final, que `aggregateDays` contaba como un día extra con una sola lectura.

Notas para la tesis (caps. 6 y 7)

Qué cambia en `documentTT` después de rehacer la app. Las secciones 00–09 de `docs/` son la fuente completa; aquí solo se dice **qué sección de la tesis queda vieja y de dónde sale el texto nuevo**, para condensarlo al pasarlo a LaTeX. Lo que toca el pipeline de datos (adquisición, Cloud SQL, vistas, Open Data) no cambia con la app; sus diferencias están en `~/Descargas/proj-files/docs/reporte_cambios_ingesta.md`.

Capítulo 6 · Arquitectura del sistema

6.1 Requerimientos

- RF-01…07 (pipeline) siguen igual.
- RF-08, RF-09 y RF-10 hablan de "la interfaz web". La app tiene ahora sus propios requerimientos por capacidad (RF-APP-01…10 y RNF-APP-01…11), que aplican igual a web y a móvil. Lo más limpio es una subsección "Requerimientos de la aplicación" con esas dos tablas condensadas, y que RF-08…10 apunten a ellas. La columna "Refina" de 01 · Requerimientos dice qué RF-APP detalla a cada RF de la tesis, y la sección "Diferencias con el cap. 6" de ese mismo documento lista lo que cambió.
- Los criterios de aceptación de cada RF-APP son verificables y tienen su prueba en la matriz de 08 · Calidad y pruebas; sirven para el cap. 8 (Resultados).

6.x Casos de uso (sección nueva)

- Diagrama UML: `diagramas/casos-de-uso.puml`. Exportar a SVG o PNG con PlantUML. Actores: Comunidad ESCOM y Estudiante/investigador (que hereda los casos de la comunidad); la consola Davis y la Cloud Function son actores de sistema. No hay actor "mantenedor": el despliegue va en el anexo.
- Especificación de cada CU (precondición, flujo principal y alternos): 02 · Casos de uso. En la tesis basta una tabla por CU.

6.2 Arquitectura general

- Agregar al diagrama de componentes los contenedores de la app: web Next.js en **Cloud Run** (con el proxy `/api/live`), app móvil Expo con **EAS Update**, y la Cloud Function de histórico consultada directo desde el cliente (CORS `*`). El diagrama C4 está en 03 · Arquitectura.
- Ya no existe el despliegue en Vercel (retirado el 24-oct-2026, repo `weather-dashboard` archivado).
- Nueva subsección **"Arquitectura de la aplicación"**: monorepo con npm workspaces, núcleo hexagonal compartido (`@tt/core`) y paquetes de presentación compartidos (`@tt/tokens`, `@tt/i18n`, `@tt/sky-art`); web y móvil solo aportan su UI y sus adaptadores de plataforma. Fuente: 03 · Arquitectura y los ADR 0001–0010 (una línea por decisión basta, con su porqué).
- El modelo de dominio (entidades, puertos, casos de uso) está como diagrama de clases en 04 · Modelo de dominio.

Capítulo 7 · Implementación

7.x Desarrollo de la interfaz de visualización (se reescribe)

Lo que dice hoy la tesis	Lo que hay ahora
Una app web; `WeatherEntity`, `GetWeatherUseCase`, `WeatherApiAdapter` con Axios	Dos apps (web y móvil) sobre `@tt/core`: `LiveReading`, `GetLiveWeather`, `HttpLiveWeatherRepository` + `mapDavisLive`, armados por `createCore` (inyección explícita, sin singletons)
Proxies de Next para el dato en vivo **y** el histórico	Solo el dato en vivo pasa por `/api/live` (URL fija, caché de 10 s, sin MAC/IP/SSID); el histórico se pide directo a la Cloud Function
Contexto global `useApp`	Providers `useCore` y `usePreferences`; preferencias (tema, idioma, unidades, tamaño de texto) en las claves `wx-*` de `localStorage`/AsyncStorage
SWR cada 5 min con revalidación al enfocar	Consulta cada 60 s y a pedido, con la vigencia del dato ("hace N s", desactualizado, sin conexión)
Rejilla bento de 1/2/4 columnas	Diseño de Claude Design: rejilla de 1/2/3 columnas en web y una columna en móvil, con 21 escenas del cielo animadas y pantalla de carga
Histórico: día específico, rango de fechas y "últimos 1–30 días" con slider	Atajos 24 h / 7 / 30 / 90 días y "últimos N días" (desde el 19-abr-2026); resolución 15 min. . . 1 mes limitada a 4–800 puntos
Caché de `localStorage` y esqueleto de carga	Caché persistente por día de CDMX (`localStorage`/AsyncStorage): solo se piden los días que faltan y los cerrados no vencen (ADR-0013); el periodo largo se pide en tramos de 10 días en paralelo (tope de 1000 filas)
`LineChart` de MUI X con interpolación catmull-Rom	Gráficas SVG propias: curva monótona que pasa por cada punto sin inventar picos, bandas mín–máx, barras de lluvia, acumulado, rosa de los vientos y cursor compartido. La geometría vive en `@tt/core` (ADR-0006)
Descarga JSON del estado y CSV	CSV, JSON o Parquet de lo que se ve (resolución y variables visibles), con la unidad en cada columna y huecos vacíos; Parquet con tipos y hora UTC para análisis de datos (ADR-0012); en móvil por la hoja de compartir

Material para reescribirla:

- Pantallas y preferencias: 07 · Interfaz. Capturas nuevas: "Ahora" y "Histórico" en web (claro y oscuro) y en móvil; reemplazan a las figuras de MUI.
- Diagramas de secuencia (dato en vivo con polling y proxy, histórico por tramos, cambio de preferencias): 05 · Flujos. Reemplazan a los diagramas de secuencia del dashboard y del pipeline en el cliente.
- Escena del cielo: la regla de 7 capas (nota `skyMap` del diseño) está en 06 · Datos; el arte se extrae del canvas (ADR-0009) y en móvil sus animaciones se reproducen como datos (ADR-0010).

7.x Calidad de los datos en la app (subsección nueva)

Las rarezas de las fuentes se corrigen en un solo lugar (los mappers de `@tt/core`), cada una con su prueba. Las que conviene contar en la tesis, porque cambian lo que el usuario ve:

- **D1:** el orden real de los extremos `hl*` (los dos mappers previos estaban mal).

- **D2/D3:** la presión del histórico viene en mmHg y, desde el 20-oct-2026, reducida a nivel del mar; las lecturas previas sin corregir se descartan al graficar.
- **D7/D9/D11:** un día de retraso, fecha + hora en hora de CDMX y extremos incluidos al consultar por rango.
- **D12:** la consola tiene la zona horaria en UTC−5; la app compensa amanecer y ocaso.

Tabla completa: 06 · Datos. D5 (`wind_dir`/`hi_dir` intercambiados en la captura) **no va en la tesis**: se corrige en el pipeline sin reportarlo (decisión del 6-nov).

7.x Mejoras después de la primera versión (fase 8, sección nueva o ampliación)

Cambios pedidos tras usar la app; cada uno tiene su ADR o su nota y sus pruebas:

- **Exportación a Parquet** (ADR-0012): además de CSV y JSON, para análisis con pandas, DuckDB o Spark. Columnas tipadas (`DOUBLE`, hora UTC como `TIMESTAMP`), huecos como `null` y la resolución y unidades en los metadatos. Se escribe en el navegador y en el teléfono con una librería de JS puro, sin servidor.
- **Caché del histórico por día** (ADR-0013): menos consultas a la Cloud Function. Los periodos se traslapan, así que solo se piden los días que faltan; los días cerrados no vencen y los recientes vencen a la hora, porque el histórico llega con un día de retraso (D7). Persiste entre sesiones.
- **Widget sin conexión** (ADR-0011, ajuste): la app y el widget guardan la última lectura completa; sin red, el widget la dibuja en gris con su hora en vez de quedar "sin conexión". Ese estado queda solo para cuando nunca hubo una lectura. Motivo: Android despierta el widget cada 30 min, muchas veces sin red (modo Doze).
- **Accesibilidad:** auditoría automática y lista manual (ver "Pruebas y despliegue").
- **Distribución del APK:** identificador definitivo `mx.ipn.escom.ttb033` y URLs como variables de entorno de EAS, fuera del repositorio (09 · Despliegue).
- **Arranque sin splash:** Android 12+ siempre muestra una pantalla de arranque del sistema; ahora va sin ícono y con el color del cielo de la pantalla de carga, así que al abrir el APK se ve directo la carga.

7.x Pruebas y despliegue

- Estrategia por capa y matriz de trazabilidad RF → CU → componente → prueba: 08 · Calidad y pruebas. Cifras al 6-nov-2026: 821 pruebas automatizadas (núcleo 362, arte 183, web 107, i18n 59, tokens 56, móvil 54) y cobertura de líneas del núcleo de 99.6 %. Van mejor en el cap. 8.
- Accesibilidad (RNF-APP-05): contraste calculado en los dos temas, auditoría axe en web (0 violaciones), revisión de rol, nombre y área táctil en móvil, y una lista de verificación manual con TalkBack (08 · Calidad y pruebas).
- CI/CD, staging por PR y producción separada para web y móvil: 09 · Despliegue. Va en un anexo, no en el cuerpo (decisión de la fase D).

Figuras que se reemplazan

Figura actual	Reemplazo
Diagrama de la arquitectura hexagonal	Componentes por capa de 03 · Arquitectura
Panel principal claro / oscuro	Capturas de "Ahora" (web y móvil, ambos temas)
Secuencia UML del dashboard	Secuencia del dato en vivo de 05 · Flujos
Controles de consulta (día, rango, últimos N días)	Captura de los controles del histórico
Secuencia del pipeline de gráficas	Secuencia del histórico por tramos de 05 · Flujos

Figura actual	Reemplazo
Gráficos históricos (MUI)	Capturas de las secciones del histórico
Flujo de descarga	Una línea: se exporta lo que se ve (`exportHistory`)
— (nueva)	Diagrama UML de casos de uso (`diagramas/casos-de-uso.puml`)